

AN INTERACTIVE PDF CHATBOT USING SEMANTIC EMBEDDINGS AND FAISS-BASED VECTOR SEARCH

*A project Report Submitted in the partial
fulfilment of the requirements for the award
of the degree*

BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING

Submitted by

DUDDU SATISH BABU (22471A05MO)
KOPPURAVURI V S N S S ASHOK KUMAR (22471A05M8)
RAJABOINA CHANDRA SEKHAR (21471A05Q2)

Under the esteemed guidance of

Syed Rizwana, M.Tech., (Ph.D)

Associate Professor



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

NARASARAOPETA ENGINEERING COLLEGE: NARASAROPET

(AUTONOMOUS)

Accredited by NAAC with A+ Grade and NBA under Tyre -1

and an ISO 9001:2015 Certified

Approved by AICTE, New Delhi, Permanently Affiliated to JNTUK, Kakinada

KOTAPPAKONDA ROAD, YALAMANDA VILLAGE,

NARASARAOPET- 522601

2024-2025

NARASARAOPETA ENGINEERING COLLEGE
(AUTONOMOUS)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that the project that is entitled with the name “**AN INTERACTIVE PDF CHATBOT USING SEMANTIC EMBEDDINGS AND FAISS-BASED VECTOR SEARCH**” is a Bonafide work done by **DUDDU SATISH BABU(22471A05M0)**, **KOPPURAVURI V S N S S ASHOK KUMAR (22471A05M8)**, **RAJABOINA CHANDRA SEKHAR(21471A05Q2)** in partial fulfilment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in the Department of **COMPUTER SCIENCE AND ENGINEERING** during 2025-2026.

PROJECT GUIDE

Sd. Rizwana, B. Tech, M. Tech., (Ph.D).
Associate Professor

PROJECT COORDINATOR

Sd. Rizwana, B. Tech, MTech., (Ph.D).
Associate Professor

HEAD OF DEPARTMENT

Dr. S. N. Tirumala Rao, M. Tech., Ph.D.
Professor & **HOD**

EXTERNAL EXAMINER

DECLARATION

We declare that this project work titled "**AN INTERACTIVE PDF CHATBOT USING SEMANTIC EMBEDDINGS AND FAISS-BASED VECTOR SEARCH**" is composed by me that the work contain here is my own except where explicitly stated otherwise in the text and that this work has not been submitted for any other degree or professional qualification except as specified.

BY

DUDDU SATISH BABU (22471A05M0)

K V S N S S ASHOK KUMAR (22471A05M8)

RAJABOINA CHANDRA SEKHAR (21471A05Q2)

ACKNOWLEDGEMENT

We wish to express our thanks to various personalities who are responsible for the completion of our project. We are extremely thankful to our beloved chairman, **Sri M. V. Koteswara Rao, B.Sc.**, who took keen interest in our in every effort throughout this course. We owe our sincere gratitude to our beloved principal, **Dr. S. Venkateswarlu, Ph.D.**, for showing his kind attention and valuable guidance throughout the course.

We express our deep-felt gratitude towards **Dr. S. N. Tirumala Rao, M.Tech., Ph.D.**, HOD of the CSE department, and to our guide, **M. Suneetha, B.Tech., M.Tech.**, Professor of the CSE department, whose valuable guidance and unstinting encouragement enabled us to accomplish our project successfully in time.

We extend our sincere thanks to **Sd. Rizwana, B.Tech., M.Tech., (Ph.D.)**, Assistant Professor & Project Coordinator of the project, for extending her encouragement. Their profound knowledge and willingness have been a constant source of inspiration for us throughout this project work. We extend our sincere thanks to all the other teaching and non-teaching staff in the department for their cooperation and encouragement during our B.Tech. degree.

We have no words to acknowledge the warm affection, constant inspiration, and encouragement that we received from our parents.

We acknowledge the encouragement received from our friends and those who were involved in giving valuable suggestions and clarifying our doubts, which really helped us in successfully completing my project.

BY

DUDDU SATISH BABU (22471A05M0)

K V S N S S ASHOK KUMAR (22471A05M8)

RAJABOINA CHANDRA SEKHAR (21471A05Q2)



INSTITUTE VISION AND MISSION

INSTITUTE VISION

To emerge as a Centre of excellence in technical education with a blend of effective student centric teaching learning practices as well as research for the transformation of lives and community.

INSTITUTE MISSION

M1: Provide the best class infrastructure to explore the field of engineering and research

M2: Build a passionate and determined team of faculty with student centric teaching, imbibing experiential, innovative skills

M3: Imbibe lifelong learning skills, entrepreneurial skills and ethical values in students for addressing societal problems



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

VISION OF THE DEPARTMENT

To become a centre of excellence in nurturing the quality Computer Science & Engineering professionals embedded with software knowledge, aptitude for research and ethical values to cater to the needs of industry and society.

MISSION OF THE DEPARTMENT

The department of Computer Science and Engineering is committed to

M1: Mould the students to become Software Professionals, Researchers and Entrepreneurs by providing advanced laboratories.

M2: Impart high quality professional training to get expertise in modern software tools and technologies to cater to the real-time requirements of the industry.

M3: Inculcate teamwork and lifelong learning among students with a sense of societal and ethical responsibilities.



PROGRAM SPECIFIC OUTCOMES (PSO's)

PSO1: Apply mathematical and scientific skills in numerous areas of Computer Science and Engineering to design and develop software-based systems.

PSO2: Acquaint module knowledge on emerging trends of the modern era in Computer Science and Engineering.

PSO3: Promote novel applications that meet the needs of entrepreneurs, environmental and social issues.

PROGRAM EDUCATIONAL OBJECTIVES (PEO's)

The graduates of the program can:

PEO1: Apply the knowledge of Mathematics, Science and Engineering fundamentals to identify and solve Computer Science and Engineering problems.

PEO2: Use various software tools and technologies to solve problems related to academia, industry and society.

PEO3: Work with ethical and moral values in the multi-disciplinary teams and can communicate effectively among team members with continuous learning.

PEO4: Pursue higher studies and develop their career in software industry.

PROGRAM OUTCOMES

PO1: Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2: Problem analysis: Identify, formulate, research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3: Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for public health and safety, and the cultural, societal, and environmental considerations.

PO4: Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5: Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6: The engineer and Society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to professional engineering practice.

PO7: Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of engineering practice.

PO8: Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings

PO9: Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO10: Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO11: Life-long learning: Recognize the need for and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROJECT SOURCE OUTCOMES (CO's):

CO421.1: Analyze the System of Examinations and identify the problem.

CO421.2: Identify and classify the

requirements. **CO421.3:** Review the

Related Literature **CO421.4:** Design and

Modularize the project

CO421.5: Construct, Integrate, Test and Implement the Project.

CO421.6: Prepare the project Documentation and present the Report using appropriate method.

Course Outcomes – Program Outcomes mapping

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
C421.1		✓										✓		
C421.2	✓		✓		✓							✓		
C421.3				✓		✓	✓	✓				✓		
C421.4			✓			✓	✓	✓				✓	✓	
C421.5				✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
C421.6								✓	✓	✓		✓		✓

Course Outcomes – Program Outcome correlation

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
C421.1	2	3										2		
C421.2			2		3							2		
C421.3				2		2	3	3				2		
C421.4			2			1	1	2				3	2	
C421.5					3	3	3	2	3	2	2	3	2	1
C421.6									3	2	1	2	3	

Note: The values in the above table represent the level of correlation between CO's and PO's:

1. Low level
2. Medium level
3. High level

Project mapping with various courses of Curriculum with Attained PO's:

Name of the course from which principles are applied in this project	Description	Attained PO
C2204.2, C22L3.2	Requirement gathering and planning for multimodal cyberbullying detection using BERT, ResNet18, and Inter-Modal Attention	PO1, PO3
CC421.1, C2204.3, C22L3.2	Analysing requirements and defining the process flow	PO2, PO3
CC421.2, C2204.2, C22L3.3	Designing architecture using UML diagrams	PO3, PO5, PO9
CC421.3, C2204.3, C22L3.2	Testing and integrating all modules	PO1, PO5
CC421.4, C2204.4, C22L3.2	Preparing project documentation	PO10
CC421.5, C2204.2, C22L3.3	Presenting work progress periodically	PO10, PO11
C2202.2, C2203.3, C1206.3, C3204.3, C4110.2	Implementing the model with scope for future enhancements	PO4, PO7
C32SC4.3	Creating prototype interface for post classification	PO5, PO6

ABSTRACT

This system presents a comprehensive intelligent document interaction system that enables natural-language querying of PDF documents through a conversational interface, with the objective of improving accessibility, efficiency, and accuracy in information retrieval from unstructured textual content. The proposed system is designed to bridge the gap between large-scale document repositories and end users by allowing intuitive question-answer interactions without requiring manual navigation or prior technical expertise. The system architecture integrates three tightly coupled technological components. First, semantic representations of document text are generated using Sentence-BERT, enabling the capture of contextual and semantic relationships between queries and document passages beyond keyword-based matching. Second, FAISS is employed to perform efficient and scalable vector similarity search, ensuring low-latency retrieval of the most relevant document segments even for large documents and growing collections. Third, a FLAN-T5-XL language model is utilized to generate coherent, context-aware responses by conditioning on the retrieved passages, thereby producing answers that are both linguistically fluent and grounded in the source material. Overall, the proposed framework provides a scalable and practical approach to intelligent document understanding and interaction. It is particularly beneficial for professionals, researchers, and general users who frequently engage with lengthy or complex PDF documents, offering a significant reduction in time and cognitive effort required to locate critical information. The system also highlights the potential of combining embedding-based semantic search with instruction-tuned large language models for advanced document-centric conversational applications.

INDEX

S.NO	CONTENT	PAGE NO
1	INTRODUCTION	1
1.1	MOTIVATION	3
1.2	PROBLEM STATEMENT	4
1.3	OBJECTIVES	5
2	LITERATURE SURVEY	7
3	SYSTEM ANALYSIS	9
3.1	EXISTING SYSTEM	9
3.1.1	DISADVANTAGES OF THE EXISTING SYSTEM	11
3.2	PROPOSED SYSTEM	12
3.3	FEASIBILITY STUDY	16
4	SYSTEM REQUIREMENTS	19
4.1	SOFTWARE REQUIREMENTS	19
4.2	REQUIREMENT ANALYSIS	19
4.3	HARDWARE REQUIREMENTS	20
4.4	SOFTWARE	20
4.5	SOFTWARE DESCRIPTION	22
5	SYSTEM DESIGN	23
5.1	SYSTEM ARCHITECTURE	23
5.1.1	DATASET	25

S.NO	CONTENT	PAGE NO
5.1.2	DATA PREPROCESSING	25
5.1.3	MODEL BUILDING	33
5.1.4	COMPARITIVE MODELS	36
5.2	MODULES	39
5.3	UML DIAGRAMS	44
6	IMPLEMENTATION	50
6.1	MODEL IMPLEMENTATION	50
6.2	CODING	50
7	TESTING	64
7.1	UNIT TESTING	64
7.2	INTEGRATION TESTING	67
7.3	SYSTEM TESTING	68
8	RESULT ANALYSIS	71
9	OUTPUT SCREENS	80
10	CONCLUSION	84
11	FUTURE SCOPE	85
12	REFERENCES	86

LIST OF FIGURES

S.NO	FIGURE DESCRIPTION	PAGE NO
1	FIG 1 Classification of Chatbots	2
2	FIG 3.1 Flowchart of existing detection systems for chatbot	10
3	FIG 3.2 Flowchart of proposed Semantic Embedding Vector Based System	13
4	FIG 5.1 Modal Architecture	23
5	FIG 5.1.1 SAMPLE DATASET IMAGES FROM MMHS150K	27
6	FIG 5.1.4.1 Flow Diagram Of PDF Chatbot Pipeline	36
7	FIG 5.3.1 UML use case Diagram for PDF chatbot system	45
8	FIG 5.3.1 UML class Diagram for PDF chatbot system	46
9	FIG 7.1 Example detection result – Cyberbullying detected	69
10	FIG 7.2 Example detection result – No cyberbullying detected	69
11	FIG 7.3 Example detection result – Invalid input	70
12	FIG 8.1 Cosine Similarity Comparison On different models	71

S.NO	FIGURE DESCRIPTION	PAGE NO
13	FIG 8.2 Semantic Similarity comparison On different models	72
14	FIG 8.3 Accuracy comparison On different models	73
15	FIG 8.4 Response Efficiency comparison on different models	74
16	FIG 8.5 Confusion matrix for proposed model	76
17	FIG 8.6 ROC curve for proposed model	78
18	FIG 9.1 Home page of prototype interface	81
19	FIG 9.2 Upload page for multimodal post detection	81
20	FIG 9.3 Classification output screen	82
21	FIG 9.4 Model evaluation screen	82
22	FIG 9.5 Project workflow screen	83

LIST OF TABLES

S.NO	FIGURE DESCRIPTION	PAGE NO
1	TABLE 1. DATASET CLASSES CHARACTERISTICS AND APPLICATIONS	28
2	TABLE 2. MODEL PERFORMANCE COMPARISON	75

1. INTRODUCTION

Social digitization of information has transformed the way people create, share, and consume knowledge. In both professional and academic settings, the movement from paper-based documents to digital archives has greatly enhanced efficiency, storage, and accessibility. Among the formats available, the Portable Document Format (PDF) has become the standard of choice. PDFs are widely appreciated because they preserve formatting consistently across platforms, ensuring that text, figures, and layouts appear the same on any device. For this reason, universities, corporations, government bodies, and legal institutions have adopted PDFs as their preferred medium for exchanging information.

Despite these advantages, PDFs remain static in nature. This means they are excellent for presenting information but extremely limited when it comes to interacting with the content. Users often face significant challenges when trying to retrieve specific information, particularly in large, unstructured, or technical documents. For example, a student may struggle to locate the precise methodology in a lengthy research paper; a lawyer may spend hours identifying relevant clauses in a legal contract; and a corporate employee may have difficulty finding the latest policy updates buried in a multi-hundred-page operations manual. While PDFs preserve structure, they lack the dynamic capabilities needed for modern information workflows.

The proposed framework integrates three core technological components:

1. **Semantic Text Embeddings** – Using **Sentence-BERT**, the text from PDF documents is converted into numerical vector representations that capture the semantic meaning of sentences and paragraphs. This ensures that conceptually similar ideas are grouped together in a high-dimensional space, even if they are worded differently.
2. **Efficient Vector Similarity Search** – Implemented with **FAISS indexing**, the system performs high-speed retrieval of the most relevant text segments. FAISS is designed to handle large-scale datasets efficiently, making it possible to query even multi-hundred- page documents in real time.
3. **Response Generation** – The retrieved text chunks are passed to the **FLAN-T5-XL language model**, which generates coherent, context-aware answers in natural language. Unlike keyword search, this approach ensures that responses are not only relevant but also presented in a way that is easy for the user to understand.

These components form a complete pipeline that transforms static PDFs into interactive, queryable knowledge sources. The system is implemented as a **web-based application** with a simple interface. Users can upload documents, ask questions in plain English, and receive direct, meaningful responses — all without requiring any technical expertise. By lowering the barrier to adoption, the chatbot makes advanced AI-driven document interaction accessible to students, professionals, and researchers alike.

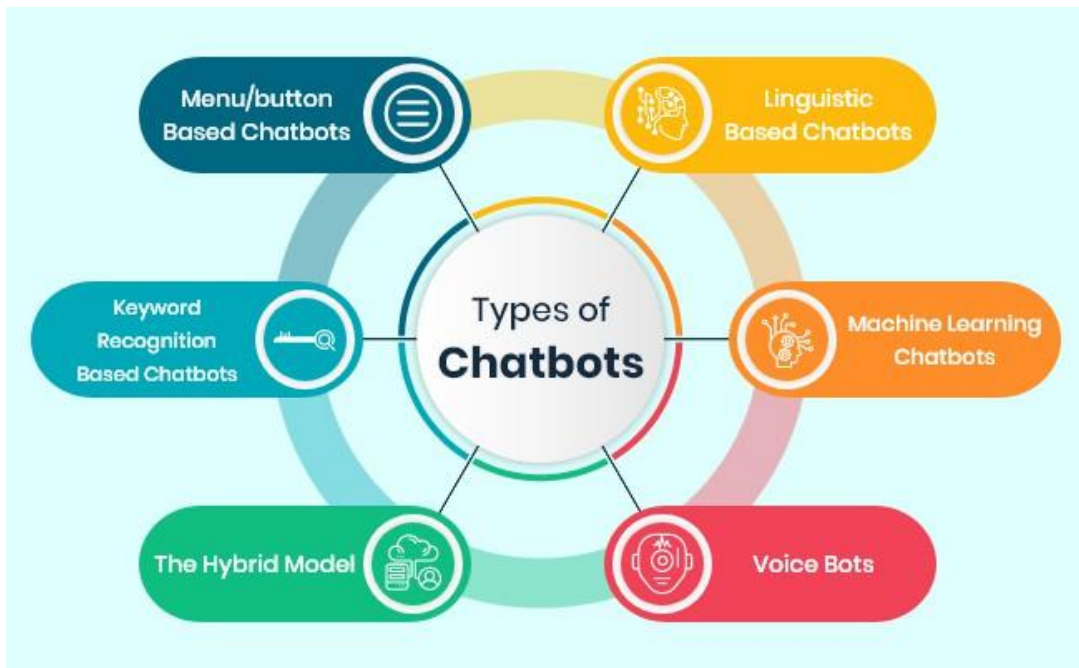


FIG 1 Classification of Chatbots

Figure 1 classifies chatbots into six categories based on technology and interaction style. **Menu/button-based chatbots** provide predefined options for simple tasks, while **keyword recognition chatbots** match queries to keywords but lack deep context understanding. **Linguistic-based chatbots** use NLP rules for grammar and syntax interpretation but require manual setup. Advanced types include **machine learning chatbots**, which learn from interactions for context-aware responses, and **voice bots**, which enable hands-free communication through speech recognition. Finally, **hybrid models** combine multiple techniques for flexibility and reliability reflecting the evolution of chatbots from rule-based systems to adaptive AI-driven solutions.

1.1 MOTIVATION

In today’s information-driven world, professionals and researchers across education, healthcare, law, and corporate sectors depend on timely access to accurate knowledge for decision-making. With the rapid pace of digitization, most of this knowledge is now stored in electronic documents, and the PDF format has become the most widely used due to its ability to preserve structure and formatting. However, despite being indispensable, PDFs remain static and lack intelligent mechanisms for information retrieval, posing real challenges for millions of users.

Traditional keyword search tools, though available in most PDF viewers, fail to bridge this gap. They depend on exact word matches and cannot understand context, synonyms, or user intent. This limitation creates inefficiencies and frustrations, especially when variations in terminology obscure relevant content.

Recent advancements in Natural Language Processing (NLP) have opened new possibilities. Semantic embeddings (e.g., Sentence-BERT) capture contextual meaning, vector indexing tools like FAISS enable high-speed semantic search, and large language models (LLMs) such as FLAN-T5-XL can synthesize natural, conversational answers. Together, these technologies allow users to interact with documents more intelligently, asking questions in plain language and receiving context-aware responses—similar to speaking with a knowledgeable assistant.

The motivation for this project lies in bridging the gap between static documents and dynamic, intelligent access. The **Interactive PDF Chatbot** aims to reduce the time and effort required to find specific information, provide an intuitive tool accessible even to non-technical users, and demonstrate the real-world value of advanced AI in domains like education, healthcare, law, and industry.

Aligned with initiatives like **Digital India**, this project contributes to making digital resources smarter, faster, and more human-friendly. Ultimately, the vision is simple yet powerful: instead of being overwhelmed by the growing sea of digital knowledge, users should be empowered by it. By transforming document interaction from passive reading into active, intelligent dialogue, this system represents a step toward truly human-centered information access.

1.2 PROBLEM STATEMENT

The widespread use of PDF documents for storing and disseminating digital information has created significant challenges in efficient knowledge retrieval. While PDFs are valued for their consistent formatting and portability, they remain inherently static and lack semantic structure. Conventional keyword-based search techniques are limited in their ability to interpret contextual meaning or complex natural language queries, often requiring users to manually scan extensive documents, which leads to reduced efficiency and increased cognitive effort.

Recent advances in Artificial Intelligence and Natural Language Processing provide effective mechanisms to overcome these limitations. Techniques such as semantic text embeddings and vector similarity search enable systems to retrieve information based on meaning rather than exact term matching, while large language models support coherent and context-aware response generation. However, these technologies are typically distributed across specialized tools and demand technical expertise, making them inaccessible to a broad range of users.

The proposed Interactive PDF Chatbot integrates these advanced techniques into a single, unified framework designed for practical usability. By combining semantic retrieval with conversational interaction, the system allows users to query document content using natural language and receive precise, source-grounded responses. This approach transforms static PDF files into intelligent, interactive knowledge sources that support efficient information discovery.

In addition to improving retrieval accuracy, the system emphasizes accessibility and scalability through a web-based interface. Users can upload documents, process them automatically, and interact with their content without requiring specialized technical knowledge. As a result, the proposed solution enhances productivity for students, researchers, and professionals, while demonstrating the effectiveness of embedding-based retrieval and language model integration for intelligent document interaction systems.

1.3 OBJECTIVES

The limitations of existing PDF search tools highlight the need for an intelligent, user-friendly system that can understand and respond to natural language queries. The overall objective of this project is to design and develop an **Interactive PDF Chatbot** that transforms static PDF documents into **dynamic, conversational knowledge sources**.

Primary Objectives

1. Enable Natural Language Querying of PDFs

- Allow users to ask questions in plain English without depending on rigid keyword searches.
- Ensure that the system interprets the *intent* behind queries rather than focusing only on exact word matches.

2. Semantic Understanding of Document Content

- Use **Sentence-BERT embeddings** to capture the meaning of sentences and paragraphs.
- Ensure that conceptually related information is retrieved even if it is expressed using different terminology.

3. Efficient and Accurate Information Retrieval

- Implement **FAISS vector similarity search** to quickly identify relevant sections of text from large and complex documents.
- Minimize user effort in navigating long files by presenting only the most contextually relevant results.

4. Generate Coherent, Human-like Responses

- Integrate the **FLAN-T5-XL language model** to synthesize retrieved information into fluent, context-aware answers.
- Present responses in a conversational style that feels natural and easy to understand.

5. Build an Accessible Web Application

- Design a simple, interactive interface that allows users to upload PDFs and ask queries without technical expertise.

- Ensure cross-platform usability so that students, professionals, and researchers can adopt the system easily.

Secondary Objectives

1. Improve User Productivity

- Reduce the time and effort needed to search through lengthy documents.
- Allow professionals to focus on decision-making rather than manual searching.

2. Support Diverse Use Cases

- Demonstrate practical utility in domains such as education, legal document analysis, corporate knowledge management, and healthcare.

3. Ensure Scalability and Performance

- Design the system to handle large documents efficiently without compromising accuracy or response quality.

4. Lay the Foundation for Future Enhancements

- Provide scope for extensions such as multilingual support, domain-specific fine-tuning, and OCR integration for scanned PDFs.

2. LITERATURE SURVEY

The domain of intelligent document interaction has gained significant attention with the rapid growth of digital information and the widespread adoption of PDF documents across academia, industry, and government sectors. Although PDFs ensure consistent formatting and reliable document preservation, their static and unstructured nature poses substantial challenges for efficient information retrieval, particularly when users require precise answers from large and complex documents [1].

Early information retrieval systems relied heavily on keyword-based techniques such as TF-IDF and BM25 [2]. While these methods are computationally efficient and simple to implement, they lack semantic understanding and fail to capture contextual relationships between queries and document content. As a result, they often miss relevant information expressed using different terminology or paraphrasing, highlighting the need for semantically aware retrieval mechanisms.

Advancements in Natural Language Processing introduced dense vector representations that capture contextual meaning. Models such as Word2Vec, GloVe, and BERT marked a significant shift toward semantic representation learning. Building upon this progress, Sentence-BERT (SBERT) enabled sentence-level embeddings that allow semantically similar text segments to be mapped closely in vector space, making semantic similarity search feasible and effective [3]. However, large-scale deployment of such embeddings requires efficient vector search mechanisms.

FAISS emerged as a scalable and high-performance solution for approximate nearest-neighbor search over high-dimensional vectors [4]. Its ability to retrieve relevant document segments in real time has made it widely adopted in applications such as recommendation systems, semantic search, and document retrieval. Concurrently, conversational AI systems evolved from early rule-based chatbots to transformer-based language models capable of contextual dialogue and question answering [5], [6].

The integration of dense retrieval with generative language models led to the development of Retrieval-Augmented Generation (RAG) frameworks [7]. These systems retrieve relevant document passages before generating responses, improving

factual grounding and response coherence. Despite their promise, applying RAG to PDF documents introduces additional challenges due to complex layouts, tables, equations, and non-linear text structures [8].

Improper parsing and segmentation can lead to inaccurate retrieval or hallucinated responses.

Several domain-specific studies demonstrate the applicability of AI-driven document assistants. Neumann et al. [9] explored chatbots in educational settings, showing improved student engagement but noting concerns related to scalability and factual reliability. Alam et al. [10] proposed a PDF-driven assistant for e-commerce project management, which proved effective within its domain but lacked generalizability. Other studies highlighted barriers faced by small and medium enterprises, including high costs, system complexity, and integration challenges [11].

Comprehensive reviews [12] emphasize persistent gaps in robust document parsing, reliable response generation, and user-friendly system design. While semantic embeddings, FAISS indexing, and large language models have significantly advanced document interaction capabilities, practical adoption remains limited by scalability, reliability, and accessibility issues.

Additionally, the reviewed literature highlights a consistent emphasis on improving retrieval accuracy and response reliability, yet comparatively less focus on end-to-end system usability and real-world deployment. Many proposed solutions demonstrate strong performance in controlled experimental settings but lack intuitive interfaces and seamless integration that would enable adoption by non-technical users. Issues such as high computational requirements, dependency on complex configurations, and limited support for diverse document types further restrict practical usage. These observations reinforce the need for a balanced approach that not only advances semantic retrieval and generative capabilities but also prioritizes accessibility, scalability, and robustness, thereby motivating the design of a unified and user-centric interactive PDF chatbot system.

Overall, the literature indicates a clear need for integrated, modular systems that combine semantic retrieval, efficient indexing, and grounded response generation in an accessible framework conversational access to PDF documents for a broad range of users.

3.SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

In most organizations and academic settings, the retrieval of information from digital documents is still heavily dependent on traditional keyword-based search engines. Common tools such as the built-in search functions in PDF readers or standard information retrieval techniques like TF-IDF (Term Frequency–Inverse Document Frequency) and BM25 are widely used. These methods are simple, fast, and effective when the user knows the exact keywords or phrases contained in the document.

However, such approaches struggle when queries are expressed in natural language or when the terminology used in the query differs from that in the source text. For example, a user searching for “image recognition methods” may miss relevant content described in the document as “visual pattern detection.” This inability to capture semantic similarity makes keyword-based methods limited in handling complex or context-aware queries.

In addition to keyword searches, some existing enterprise systems integrate rule-based chatbots. Early conversational agents such as ELIZA relied on predefined patterns and scripts to simulate dialogue. Although they demonstrated the feasibility of automated conversation, they lacked depth, adaptability, and contextual reasoning. Modern rule-based document assistants can retrieve specific responses from structured knowledge bases, but they fail when dealing with lengthy, unstructured PDF files.

Another limitation in existing systems is their static nature. Traditional PDF readers allow searching, highlighting, and bookmarking but provide no interactive mechanism for querying documents conversationally. Users must manually scan through large documents, which is time-consuming and inefficient, particularly when handling technical manuals, research papers, or legal documents where precise information retrieval is critical.

Finally, while some recent systems have experimented with machine learning–based text classification or information extraction, they often lack end-to-end integration. Most tools are either too specialized for a particular domain (e.g., legal contracts, e-commerce documents) or require significant technical expertise to configure and use.

In summary, the **existing systems** provide basic support for searching and interacting with documents but remain limited by their reliance on exact keyword matching, lack of semantic understanding, minimal interactivity, and poor scalability. These shortcomings highlight the need for a more advanced, user-friendly solution — one that can **understand natural language queries, capture semantic meaning, and deliver accurate responses conversationally**.

Existing System

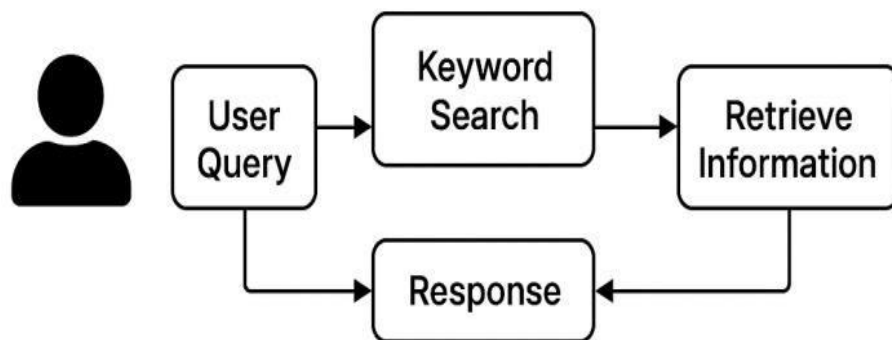


FIG 3.1 Flowchart of Existing Detection Systems for Chatbot

The above flowchart illustrates the functioning of the existing chatbot system, which primarily relies on keyword-based search mechanisms. In this process, the user inputs a query, and the system performs exact word matching to retrieve results from the document. While this approach can provide relevant results when the keywords align perfectly, it often fails to capture the context or meaning of the query. As a result, users may encounter incomplete or irrelevant outputs, leading to inefficiencies and frustration. This limitation highlights the inherent drawback of traditional keyword-driven systems, which lack the ability to understand semantics or interpret natural language effectively.

3.1.1 DISADVANTAGES OF THE EXISTING SYSTEM

Although keyword-based search engines and basic PDF readers provide a convenient way to locate words within documents, they suffer from several significant limitations:

1. Dependence on Exact Keywords

- Existing systems retrieve results only if the query exactly matches the wording used in the document.
- Synonyms, paraphrased expressions, or alternate terminologies are ignored, leading to incomplete or irrelevant outputs.

2. Lack of Semantic Understanding

- Current approaches operate at a purely lexical level without understanding context or meaning.
- For example, a search for “visual recognition” may fail to detect content described as “image pattern detection,” even though both refer to the same concept.

3. Poor User Interactivity

- Most PDF readers simply highlight keyword matches and place the burden of interpretation on the user.
- There is no conversational interface to refine, clarify, or expand queries in natural language.

4. Time-Consuming for Large Documents

- For lengthy research papers, legal contracts, or technical manuals, users must manually sift through dozens of occurrences of a keyword.
- This makes information retrieval inefficient and prone to errors.

5. Limited Adaptability Across Domains

- Rule-based or keyword systems are often restricted to specific document types.
- They fail to scale effectively across diverse fields such as academia, healthcare, law, and corporate documents without manual customization.

6. Barriers for Non-Technical Users

- Existing tools assume that users can construct precise keyword queries.
- Non-specialist users, such as students or professionals outside IT, may struggle to extract meaningful insights from complex or unstructured.

3.2 PROPOSED SYSTEM

To overcome the limitations of existing keyword-based approaches, the proposed system introduces an intelligent document interaction framework that allows users to query PDF content in natural language through a conversational interface. Instead of treating the document as static text, the system employs advanced semantic processing and retrieval-augmented generation techniques to deliver precise, context-aware responses.

Proposed System

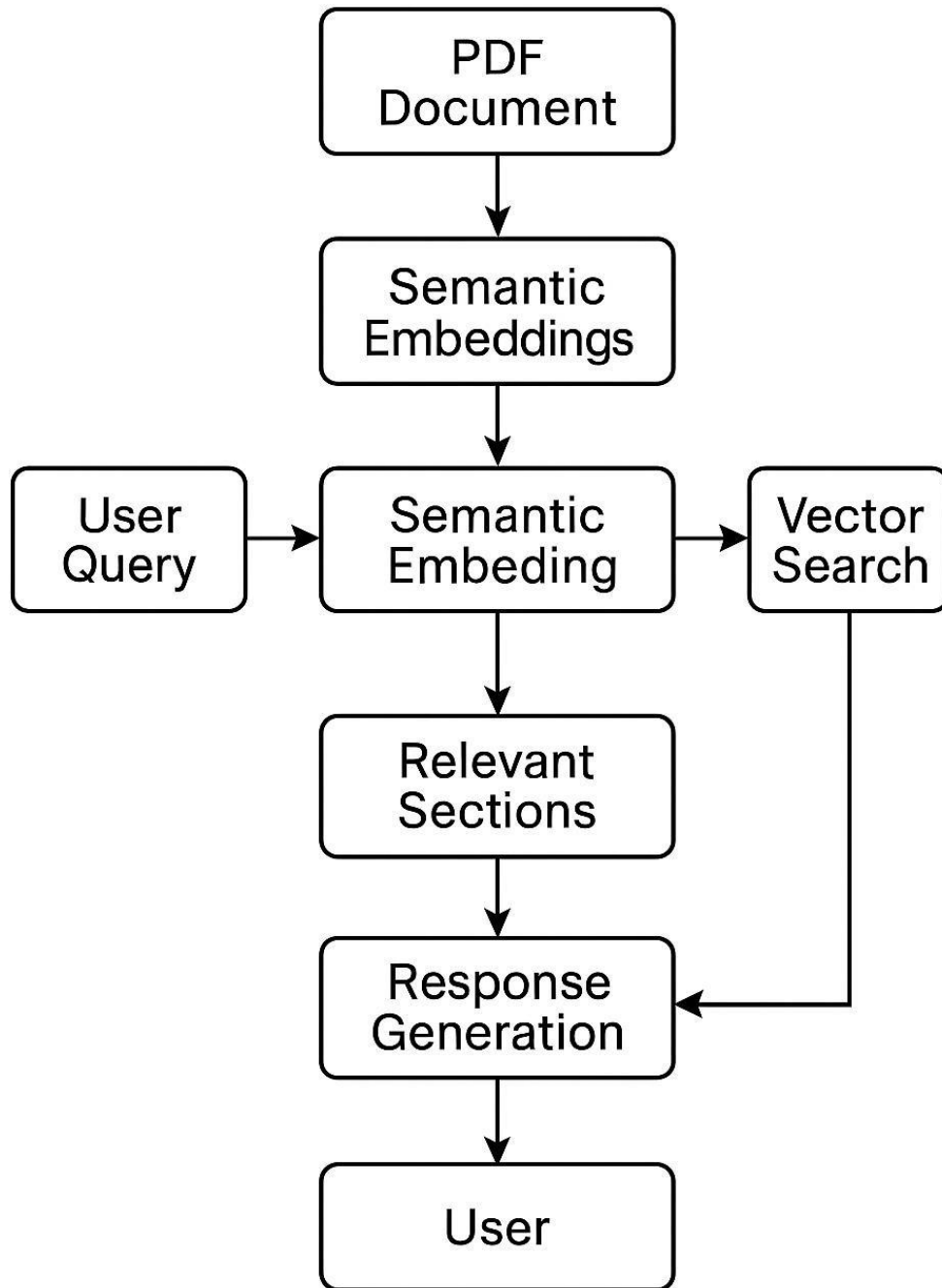


Fig 3.2 Flowchart of Proposed Semantic Embedding Vector Based System

The above Fig flowchart illustrates the **Proposed System** for an Interactive PDF Chatbot, where PDF documents are first uploaded and processed through text extraction and preprocessing modules. The extracted data is then semantically embedded and indexed using FAISS for efficient similarity search. When a user inputs a natural language query, the system retrieves the most relevant chunks and forwards them to a response generation module powered by a large language model. Finally, the chatbot provides a clear, context-aware answer through an intuitive conversational interface, ensuring fast and accurate information retrieval from static PDF documents.

Advantages of the Proposed System:

The proposed **Semantic Embedding Vector Based System Deep Learning framework** offers several advantages over traditional and existing systems:

1. Semantic Understanding of Queries

- Unlike traditional keyword search, the proposed system interprets the meaning of queries.
- This enables retrieval of relevant information even when the wording in the query and the document differs.

2. Interactive and User-Friendly

- The chatbot offers a conversational interface that allows users to ask questions in natural language.
- This reduces dependency on query syntax or technical expertise.

3. Efficient Information Retrieval

- FAISS-based vector indexing ensures fast and accurate search across large volumes of data.
- Users save time compared to manual scrolling or keyword-based search in lengthy PDFs.

4. Context-Aware Responses

- Advanced models such as FLAN-T5-XL generate coherent and context-sensitive answers.
- Users receive complete and meaningful explanations rather than isolated keyword matches.

5. Scalability Across Domains

- The pipeline is adaptable to various document types including academic, legal, business, and healthcare.
- This makes it suitable for widespread adoption across industries.

6. Reduced Cognitive Load

- Relevant passages are summarized and presented, minimizing user effort in document navigation.
- This helps students and professionals focus on understanding content rather than searching for it.

7. Accessibility for Non-Technical Users

- Developed as a web application, the system is intuitive and easy to use for all users.
- It democratizes access to intelligent document interaction tools.

In conclusion, the proposed system offers significant advantages over traditional keyword-based approaches by combining semantic understanding, interactive design, and efficient retrieval mechanisms. Its ability to generate context-aware responses, adapt across multiple domains, and reduce user effort makes it a powerful tool for professionals, researchers, and non-technical users alike. By providing accurate, meaningful, and accessible information through a user-friendly interface, the system not only enhances productivity but also democratizes intelligent document interaction, bridging the gap between static content and dynamic, human-like communication.

3.3 FEASIBILITY STUDY

A feasibility study is carried out to determine whether the proposed system is practical, achievable, and beneficial compared to existing methods. It ensures that the system can be successfully developed, deployed, and maintained without excessive costs or complexity. For the Interactive PDF Chatbot system, three key dimensions of feasibility have been evaluated: Technical Feasibility, Economic Feasibility, and Operational Feasibility.

1. Technical Feasibility

The technical feasibility focuses on evaluating whether the proposed Interactive PDF Chatbot can be developed and deployed effectively using the available hardware, software, and frameworks.

- The system utilizes Sentence-BERT for semantic text embedding and FAISS for efficient vector similarity search. Both are proven technologies, widely supported by Python libraries and frameworks such as Hugging Face Transformers and Facebook AI's FAISS library.
- For response generation, the system integrates FLAN-T5-XL, an advanced language model capable of producing context-aware and coherent answers. The model can be fine-tuned and deployed efficiently on GPUs using frameworks like PyTorch and TensorFlow.
- Preprocessing tasks such as PDF text extraction, cleaning, segmentation, and tokenization can be achieved with lightweight tools like pdfplumber, PyPDF2, and NLTK, ensuring smooth data handling.
- The system architecture is designed for modularity and scalability, making it suitable for cloud platforms such as AWS, GCP, or Azure, as well as on-premise deployment in academic or corporate environments.
- Since the hardware requirements are minimal (standard CPU/GPU-enabled systems), the solution can be scaled for both small-scale usage (individual researchers or students) and enterprise-level deployments.

Thus, the system is technically feasible, as it relies on accessible hardware, widely

supported open-source frameworks, and established NLP/AI technologies, ensuring smooth implementation and future extensibility.

2. Economic Feasibility

The Economic feasibility examines whether the proposed system is cost-effective and sustainable in terms of development, deployment, and long-term usage.

- The system primarily relies on open-source frameworks and libraries such as PyTorch, TensorFlow, Hugging Face, FAISS, and pdfplumber, which eliminates the need for costly proprietary software licenses.
- Initial development costs are minimized since the project can be built and tested in environments like Google Colab Pro or other cloud-based platforms that offer free or low-cost GPU/TPU resources.
- Deployment on cloud services (AWS, GCP, Azure) may involve subscription costs, but these are scalable and can be adjusted based on actual usage (pay-as-you-go model).
- Hardware requirements are modest, and the system can run on standard mid-range laptops/PCs or inexpensive cloud instances, reducing capital investment.
- Maintenance costs remain low as the system is modular, making updates or improvements to specific components (e.g., embeddings or response generator) possible without overhauling the entire pipeline.

Thus, the system is economically feasible, offering a high return on investment by saving users' time, improving productivity, and reducing the manual effort required for information retrieval.

3. Operational Feasibility

Operational feasibility assesses whether the system can be practically used and accepted in real-world environments.

- Ease Operational feasibility evaluates whether the system can be practically adopted and effectively used by its intended audience.
- The chatbot interface is designed to be user-friendly and conversational,

requiring no technical expertise. Users can simply upload a PDF and ask questions in natural language.

- The system reduces the cognitive load by summarizing relevant passages instead of forcing users to manually search through documents, enhancing efficiency in real-world scenarios.
- It is adaptable to multiple domains such as education, law, healthcare, and corporate documentation, making it useful for a wide variety of users including students, professionals, and researchers.
- Since it is implemented as a web application, it is accessible across devices and platforms, ensuring ease of adoption without requiring complex installations.

4. SYSTEM REQUIREMENTS

4.1 SOFTWARE REQUIREMENTS

The following software components are required for development and deployment:

- **Operating System:** Windows 10 +
- **Hardware Accelerator:** GPU/TPU
- **Programming Language:** Python 3.8+
- **Development Environment:** Jupyter Notebook / Google Colab / VS Code
- **Browser:** Any Browser with Latest Version

4.2 REQUIREMENT ANALYSIS

Functional Requirements

1. The system should allow users to upload PDF documents as input.
2. The system should extract text content from uploaded PDFs, handling different structures like tables, paragraphs, and headings.
3. The system should preprocess the extracted text (tokenization, stopword removal, segmentation) to prepare it for semantic embedding.
4. The system should generate semantic embeddings using a pretrained model (e.g., Sentence-BERT).
5. The system should index embeddings using FAISS vector search to enable fast and efficient similarity queries.

Non-Functional Requirements

1. **Performance:** The system should provide responses within a few seconds, even for large documents, while maintaining high accuracy.
2. **Scalability:** The architecture should support deployment on cloud infrastructure to handle multiple users and large document collections.
3. **Security:** All user-uploaded documents should be processed securely, ensuring privacy and compliance with data protection policies
4. **Maintainability:** The modular design should allow easy updates to embeddings, vector indexing, or language models without rewriting the entire system.

5. **Portability:** The system should run across different environments, including local machines, cloud platforms (AWS, GCP, Azure), and institutional servers.

4.3 HARDWARE REQUIREMENTS

For training and deployment, the following hardware resources are required:

- **Processor:** Intel i7 / AMD Ryzen 7 or higher
- **RAM:** Minimum 16 GB (32 GB recommended)
- **GPU:** NVIDIA RTX 4050 or higher (for local) / Tesla T4 / A100 (for cloud training)
- **Storage:** At least 200 GB for dataset (MMHS150K) and model checkpoints.

4.4 SOFTWARE

The proposed **Interactive PDF Chatbot** relies on a focused stack of languages, frameworks, and libraries that cover the full pipeline—from PDF ingestion and preprocessing to semantic search, answer generation, and web delivery. Each component below maps cleanly to a stage in the system so development, training, and deployment remain smooth and maintainable.

1. **Operating System**

- Windows 10 +
- Stable base for Python, CUDA/GPU drivers, and build tools.

2. **Programming Language: Python 3.10+**

- Python is chosen because of its rich ecosystem in artificial intelligence and machine learning.
- Its extensive library support (NumPy, Pandas, Scikit-learn, etc.) makes it the standard choice for data-driven research.

3. **Deep Learning Framework: PyTorch**

- The chatbot leverages PyTorch to fine-tune **Sentence-BERT** for semantic embeddings and to run **FLAN-T5-XL** for response generation.
- It provides dynamic computation graphs, GPU acceleration, and strong community support.

4. NLP Libraries

- **Hugging Face Transformers:** Provides pre-trained models like BERT for text embeddings.

5. Semantic Embeddings & Transformers

- Hugging Face Transformers + Sentence-Transformers
- Provides pretrained Sentence-BERT models for high-quality semantic embeddings

6. Data Handling Libraries

- **NumPy & Pandas:** For handling structured and unstructured data efficiently.
- **Scikit-learn:** Used for data preprocessing, evaluation metrics, and baseline comparisons.

7. Visualization Tools

- **Matplotlib & Seaborn:** For generating plots such as loss curves, accuracy comparisons, and confusion matrices.
- **Scikit-learn** for evaluation metrics (precision/recall, MRR, nDCG).

8. Development Environment

- **Jupyter Notebook / Google Colab:** For interactive coding, experimentation, and running GPU-based training.
- **VS Code:** For structured project development and version control.

9. Version Control

- **Git & GitHub:** For tracking project versions, collaborative development, and storing source code.

10. Deployment Tools (Optional)

- **Flask / FastAPI:** To serve the trained model as an API for real-time cyberbullying detection.
- **Cloud Platforms (AWS, GCP, Azure):** For scalable deployment, ensuring the system can monitor millions of social media posts daily.

11. Version Control & Project Management

- **Git & GitHub/GitLab** for source control, issues, and CI.

- *Optional*: **pre-commit** hooks for formatting (black/isort).

12. Packaging & Deployment

- **Docker** for reproducible builds; **docker-compose** for multi-service stacks (API + UI).
- **Nginx** as reverse proxy **HTTPS** via Let's Encrypt.
- **Weights & Biases** or **MLflow** (*optional*) for model/version tracking.

4.5 SOFTWARE DESCRIPTION

The software foundation of the Interactive PDF Chatbot has been carefully selected to balance efficiency, scalability, and user accessibility. At its core, the system is powered by **Python**, which serves as the primary programming language due to its simplicity, flexibility, and rich ecosystem of AI libraries. Python enables seamless integration of tools for natural language processing, machine learning, and web application development. Frameworks like **PyTorch** and **Hugging Face Transformers** provide state-of-the-art models such as Sentence-BERT for semantic embeddings and FLAN-T5 for response generation, ensuring high-quality natural language interaction with PDF content.

To enable fast and accurate information retrieval, the system incorporates **FAISS (Facebook AI Similarity Search)**, which is optimized for vector similarity search across large datasets. FAISS plays a vital role in matching user queries with semantically relevant text chunks extracted from PDFs, making the system far superior to traditional keyword search engines. Additional support libraries like **NumPy**, **Pandas**, and **Scikit-learn** assist in data preprocessing, feature handling, and evaluation. These libraries ensure smooth processing of structured and unstructured data, making the system reliable for both small and large-scale document collections.

For deployment and user interaction, the system leverages modern web development frameworks. **Flask** acts as the backend engine, managing API requests and integrating the AI pipeline with the user interface. On the frontend, **HTML5**, **CSS3**, and **Bootstrap** provide a clean, responsive interface that is accessible across devices and browsers. Together, these tools not only make the chatbot technically sound but also ensure that it is **intuitive, user-friendly, and accessible to non-technical users..**

5. SYSTEM DESIGN

5.1 SYSTEM ARCHITECTURE

The architecture of the proposed **Multimodal Cyberbullying Detection System** consists of multiple stages, where both textual and visual inputs are processed separately and then fused through an **Inter-Modal Attention mechanism** for final classification.

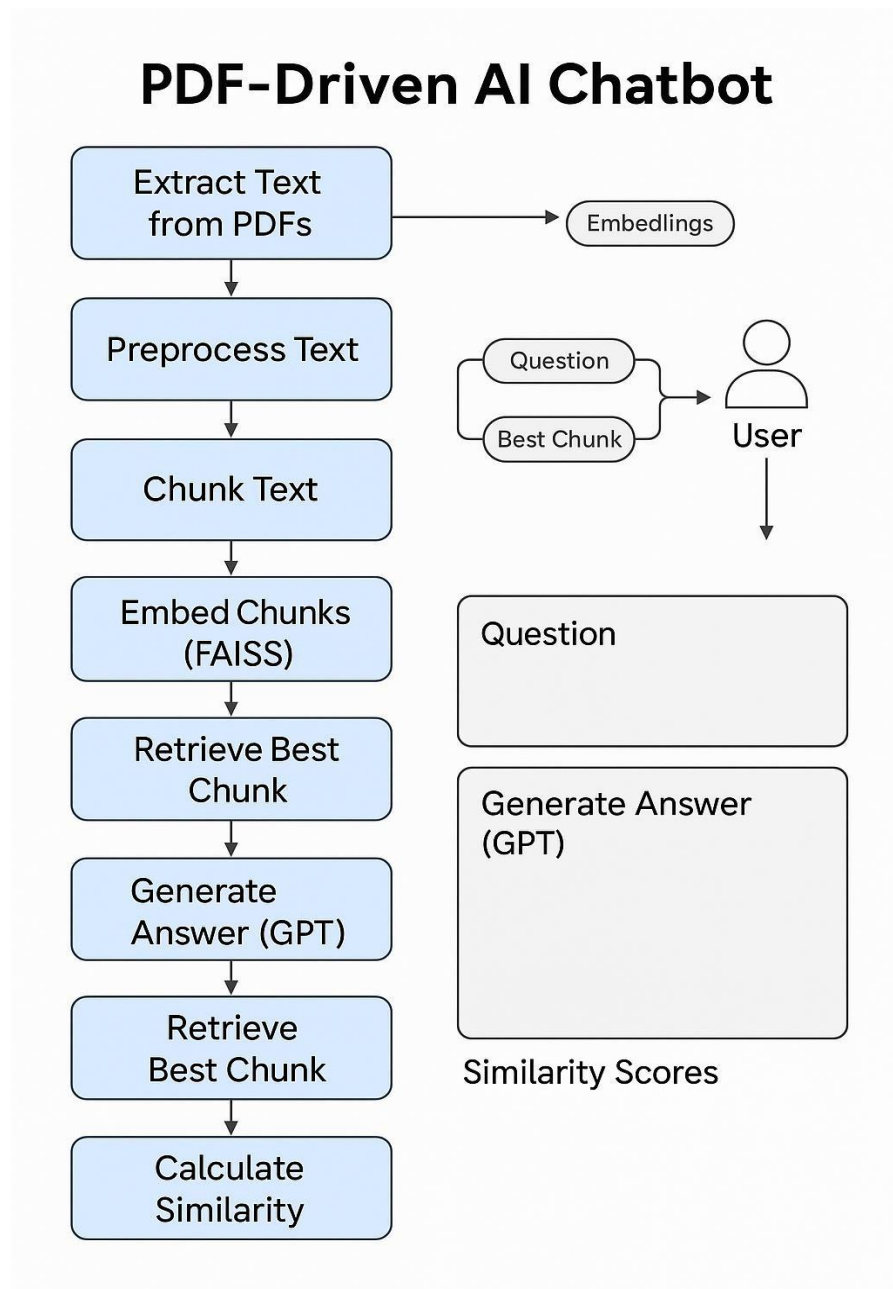


FIG 5.1 Modal Architecture

DESCRIPTION OF THE ARCHITECTURE

1. **Input Layer:** The system accepts input in the form of PDF documents along with user queries. PDFs provide the source of knowledge, while queries represent the information users wish to retrieve.
2. **Preprocessing:**
 - **Text Preprocessing:** Extracted text from PDFs undergoes cleaning, normalization, and removal of noise (headers, footers, unnecessary symbols).
 - **Chunking:** The pre-processed text is divided into smaller, manageable chunks to ensure efficient embedding and retrieval.
3. **Feature Extraction:**
 - **Embedding Generation:** Each text chunk is converted into high-dimensional vector embeddings using Sentence-BERT.
 - **Query Embedding:** User queries are also embedded in the same semantic space to enable accurate similarity matching.
4. **Similarity Search (Retrieval Layer):**
 - The embeddings are stored and indexed using FAISS, which performs rapid similarity search.
 - The system identifies the most relevant chunk(s) of text corresponding to the user's query.
5. **Answer Generation Layer:**
 - The best-matched chunks are passed to a generative language model (e.g., GPT/FLAN-T5-XL).
6. **Output & Layer:**
 - The final response is displayed to the user in natural language, along with similarity scores to ensure transparency in the retrieval process.

5.1.1 DATASET

In this project, a custom dataset is used where users can upload any PDF file, including research papers, manuals, legal documents, or academic notes. The system processes these files through text extraction, cleaning, and segmentation, then generates semantic embeddings for intelligent retrieval. This dynamic dataset approach ensures flexibility, scalability, and practical usability, enabling the chatbot to handle diverse document types and deliver context-aware responses.

DESCRIPTION OF THE DATASET

- **Name:** Custom PDF Document Dataset
- **Type:** User-Generated PDF Dataset (Textual Documents)
- **Size:** Dynamic (depends on user uploads; can range from a few pages to hundreds per document)
- **Modalities:**
- **Text:** Extracted from PDF files including research papers, technical manuals, legal contracts, and reports.
- **Metadata:** Page numbers, section headers, and document structure used for contextual indexing.
- **Annotations:** Automatically segmented into semantic chunks for indexing; enriched with embeddings for similarity-based retrieval.
- **Languages:** Primarily English, though the system can process multilingual content depending on the document.

5.1.2 DATASET PREPROCESSING STEPS

1. PDF Text Preprocessing

- Extract raw text using PDF parsers and remove headers, footers, and special characters.
- Apply sentence segmentation, followed by tokenization, stop word removal, lowercasing, and lemmatization for text normalization
- Convert cleaned text into semantic embeddings using Sentence-BERT.

2. Query Preprocessing

- Tokenize and Normalize Queries.
- Generate Embeddings with Sentence-BERT.
- Handle synonyms and phrasing variations.

3. Indexing and Storage

- Store and index embeddings with FAISS for fast retrieval, along with metadata like page numbers and section headers for context.

DATASET CHALLENGES

- **Unstructured Layouts:** PDFs often contain headers, footers, tables, or figures that disrupt clean text extraction, requiring careful preprocessing.
- **Inconsistent Formatting:** Variations in fonts, encodings, and scanned PDFs with OCR errors make extraction less reliable.
- **Domain Diversity:** Since users may upload research papers, legal files, or manuals, the system must handle varied writing styles and terminologies.

DATASET CLASSES

The custom PDF dataset is organized into the following classes and structural components for efficient retrieval and analysis:

1. Introduction / Abstract

- Contains the opening section of the PDF, summarizing the overall content.
- Example: Abstracts in research papers or executive summaries in reports.

2. Body / Main Content

- The core section of the PDF, divided into paragraphs, sections, or chapters.
- Example: Literature review, methodology, or detailed explanations in manuals.

3. Tables and Figures

- Structured data, charts, or diagrams embedded in the PDF.
- Example: Statistical tables in research papers or workflow diagrams in project reports.

4. References / Bibliography

- Lists of cited works, sources, or supporting documents at the end of the file.

5. Metadata / Miscellaneous

- Includes headers, footers, page numbers, or non-core text elements.

Final Mapping for This Project

For effective organization and retrieval, the dataset categories can be grouped into:

- **Relevant Information (Positive Class)** → Extracted meaningful content such as key paragraphs, sections, tables, or figures.
- **Non-Relevant Information (Negative Class)** → Redundant elements such as headers,
- This allows the system to function in two modes:
- **Binary classification (Relevant vs. Non-Relevant content).**
- **Multi-class classification (Specific components like Abstract, Methodology**

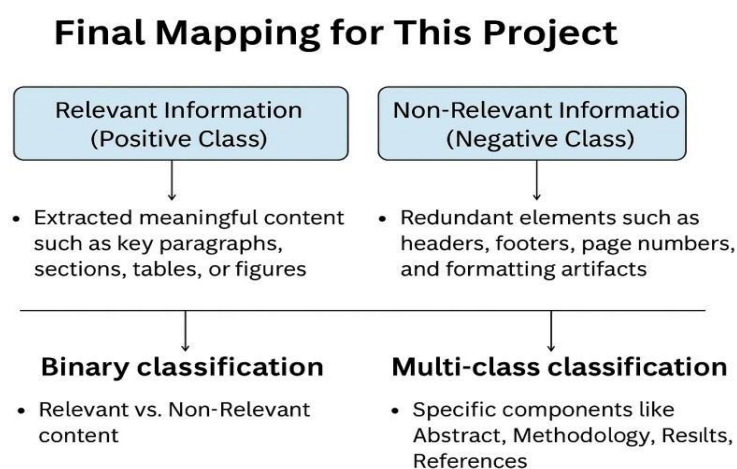


FIG 5.1.1 SAMPLE DATASET IMAGES FROM MMHS150K

Class	Image characteristics	Applications
Non-Query / Informational PDF	Well-structured research paper, manual, or report with headings, paragraphs, and figures.	Used to train the system to recognize standard documents and provide context-aware search responses.
Query-Based / Interactive PDF	Uploaded PDFs containing questions, FAQs, or instructional notes.	Helps the chatbot identify question–answer style data

TABLE 1: DATASET CLASSES CHARACTERISTICS AND APPLICATIONS

DATASET PREPROCESSING

Preprocessing plays a vital role in making the uploaded PDF dataset suitable for semantic search and chatbot responses. Since PDFs often contain unstructured text, metadata, and formatting elements, systematic preprocessing ensures clean, consistent, and meaningful input for embedding models. The following steps were applied:

Data Cleaning

- Removed duplicate tweets and images to avoid bias.
- Filtered out corrupted or missing image files.
- Handled corrupted or unreadable PDF pages during extraction.

2. Text Preprocessing

- Extracted raw text from PDFs using pdf plumber or similar parsers.
- Normalized text by lowercasing, removing special symbols, and handling Unicode characters.
- Converted cleaned text into **semantic embeddings** using Sentence-BERT for vector indexing.

Example:

Page Content → [CLEANED TEXT] → [VECTOR EMBEDDING]

3. Query Preprocessing

- Tokenized and normalized user queries for consistency.
- Generated semantic embeddings for queries using the same Sentence-BERT model.
- Aligned queries with stored embeddings to enable context-aware similarity search.

4. Indexing and Storage

- Stored processed document embeddings in **FAISS** for efficient similarity-based retrieval. Maintained metadata such as **page numbers, headings, and sections** for contextual responses. Enabled fast mapping between user queries and relevant document passages.

5. Data Preparation

- **Segmentation:** Long text passages are split into smaller chunks (e.g., 200–300 tokens) to improve retrieval accuracy.
- **Embedding Generation:** Each chunk is transformed into a high-dimensional vector using Sentence-BERT.
- **Metadata Association:** Each vector is tagged with additional details such as document title, section heading, and page number for context.
- **FAISS Indexing:** All embeddings are stored in a vector database (FAISS) to enable fast similarity search during queries.

6. Final Check

- **Integrity Check:** Verifies that all PDF pages are processed without missing or corrupted text.
- **Embedding Validation:** Confirms that every text chunk has a corresponding vector representation.
- **Query Testing:** Sample queries are run against the index to ensure meaningful and context-aware retrieval.
- **Scalability Review:** Confirms that the dataset can handle large documents and multiple concurrent queries without performance loss.

PDF TEXT PREPROCESSING RESULTS

1. Before Preprocessing

- Extracted raw text included headers, footers, page numbers, and special symbols.
- Sentences were unstructured, with inconsistent spacing and line breaks.
- Presence of noisy elements such as hyperlinks, references, and repeated formatting tags.
- Different encoding styles across PDFs caused variations in character representation.

2. After Preprocessing

- Clean text obtained by removing headers, footers, page numbers, and unwanted symbols.
- Sentences segmented and tokenized for structured representation.
- Stop words removed, text normalized to lowercase, and lemmatization applied.
- Converted into **semantic embeddings** using Sentence-BERT for efficient vector search.
- Stored with metadata (page number, section heading) for context-aware retrieval.

TEXT PREPROCESSING RESULTS

1. Before Preprocessing

- Extracted text from PDFs included unwanted elements such as headers, footers, page numbers, special characters, and inconsistent formatting.
- Sentences were fragmented across lines due to PDF formatting, making contextual understanding difficult.
- Redundant white spaces, irregular casing, and symbols were present.

2. After Preprocessing

- Removed headers, footers, page numbers, and special characters.
- Normalized text to lowercase for consistency.
- Segmented text into structured sentences and chunks suitable for embedding.

STEP	Before PreProcessing(RAW)	After PreProcessing (Cleaned & Structured)
EXTRACTED PDF TEXT	“CHAPTER 1 INTRODUCTION 1 This study aims to Develop a CHATBOT for PDF FILES !!! Page 12”	“This Study aims to develop A chatbot for pdf files””

5.1.2 Feature Extraction

In the proposed PDF-driven chatbot framework, features are extracted from textual data to enable efficient semantic search and context-aware response generation. The extracted features are mapped into a vector space for similarity-based retrieval and further processing with language models.

1. Text Feature Extraction (Sentence-BERT)

- Raw text from uploaded PDFs is segmented into sentences and tokenized.
- Each sentence is transformed into contextual embeddings using Sentence-BERT.
- From the final layer, a fixed-size sentence embedding (768-dimensional vector) is obtained.
- **Formula:**

$$h_{text} = S(S) \in R^{768}$$

where S is the input sentence.

- These embeddings are further projected into a lower-dimensional latent space (e.g., 256-dim) for efficient FAISS indexing:

$$f_{text} = W_t h_{text} + b_t, f_{text} \in \mathbb{R}^{256}$$

2. Query Feature Extraction

- User queries are pre-processed (tokenization, normalization) and passed through the same Sentence-BERT encoder.
- This ensures both document embeddings and query embeddings are represented in the same semantic space.
- The similarity between query embeddings and document embeddings is calculated using cosine similarity.

$$\text{Sim}(Q, D) = \frac{Q \cdot D}{\|Q\| \|D\|}$$

Fusion & Retrieval

- FAISS indexes document embeddings and retrieves the top-k most similar chunks to a user query.
- The retrieved embeddings are combined with query embeddings to provide context-aware input to the language model (e.g., FLAN-T5).
- This ensures that the generated response is relevant, coherent, and grounded in the source PDF.

Key Features Extracted

- **FromTextChunks:**

The system captures deep semantic representations of PDF content, going beyond surface-level keywords. It identifies contextual meaning, paraphrased expressions, domain-specific terminology, and technical knowledge embedded in research papers, manuals, or legal documents. This enables accurate retrieval even when user queries use different wording than the source text.

- **FromQueries:**

User queries are transformed into semantic embeddings that capture intent, synonyms, variations in phrasing, and contextual alignment with document content. The system recognizes subtle differences in query structure and adapts by mapping them to conceptually relevant passages. This ensures that even

vague or indirect questions can be matched to the correct section of the PDF.

- **FromFusion:**

By combining query embeddings with document embeddings, the system learns to identify the most relevant text segments. This fusion ensures accurate similarity matching, prioritizing content that best addresses the user's information need.

5.1.3.MODEL BUILDING

Our implementation integrates **three core components**:

1. Sentence-BERT (SBERT) for generating semantic embeddings of text chunks from PDFs.
2. FAISS (Facebook AI Similarity Search) for efficient indexing and retrieval of the most relevant text chunks.
3. Generative Language Model (e.g., FLAN-T5 or GPT) for producing coherent, context-aware answers based on retrieved content

1. Sentence-BERT for Textual Feature Extraction

PDF text is often unstructured, lengthy, and domain-specific, which makes bag-of-words or TF-IDF methods ineffective. To overcome this, we use Sentence-BERT (all-MiniLM-L6-v2), a transformer-based model optimized for semantic similarity.

- **Input:** Preprocessed text chunks extracted from PDF documents.
- **Processing:** Each chunk is tokenized, encoded by the transformer layers, and mean-pooled to form a dense vector representation.
- **Output:** A 384-dimensional embedding vector representing the semantic meaning of the chunk.

These embedding captures paraphrasing, technical context, and semantic similarity, enabling robust alignment between user queries and PDF text.

2. FAISS for Semantic Retrieval

Since large PDFs may contain thousands of text chunks, efficient retrieval is essential. We employ FAISS (Facebook AI Similarity Search) for approximate nearest neighbor (ANN) search.

- **Input:** User query embeddings generated by Sentence-BERT.
- **Processing:** Cosine similarity between query embeddings and stored chunk embeddings is computed.
- **Output:** Top-k most relevant chunks, ranked by similarity score. This ensures that even in large datasets, the chatbot can quickly retrieve passages most relevant to the user's query.

3. FLAN-T5 for Generative Answer Construction

Once the relevant chunks are retrieved, FLAN-T5, an instruction-tuned generative model, constructs a natural language answer.

- **Input:** User query + top-k retrieved chunks.
- **Processing:** The encoder-decoder transformer processes the context and query to produce a coherent, context-aware response.
- **Output:** A natural language answer, grounded in the retrieved document content.

This enables the system to generate not just extracts but well-formed explanations.

4. Classification Layer

For tasks like relevance verification, a classifier can be added on top of embeddings to assess whether retrieved content sufficiently answers the query.

- **Output:** A probability score reflecting relevance.
- **Decision Rule:**
 - $\geq 0.5 \rightarrow$ Relevant passage.
 - $< 0.5 \rightarrow$ Low-confidence result (fallback to clarification request).

5. Training & Fine-Tuning Strategy

To optimize performance, fine-tuning is applied on domain-specific PDFs when needed.

- **Loss Function:** Contrastive loss for embedding alignment, cross-entropy for classification.
- **Optimizer:** AdamW with learning rate scheduling.
- **Batch Size:** 16–32, depending on GPU resources.
- **Evaluation Metrics:** Accuracy, Recall@k, and BLEU/ROUGE for generative quality.
- **Checkpoints:** Best-performing weights stored for deployment.

6. Advantages of the Proposed Model

Our architecture offers several benefits over traditional PDF search and QA systems:

1. **Semantic Understanding:** Captures meaning beyond keywords, handling paraphrased or indirect queries.
2. **Efficient Retrieval:** FAISS ensures scalability for large PDF repositories.
3. **Human-like Answers:** FLAN-T5 generates fluent, explanatory responses instead of raw extracts.
4. **Domain Adaptability:** Works across varied PDFs—legal, academic, technical—without retraining.

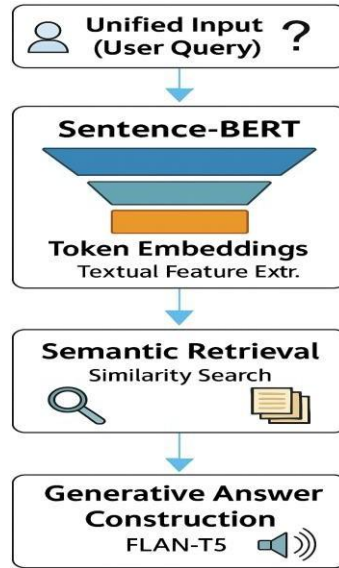


FIG 5.1.4: Flow Diagram of PDF Chatbot Pipeline – Sentence-BERT encodes the user query into embeddings, FAISS performs semantic retrieval, and FLAN-T5 generates the final natural language response.

This figure illustrates the end-to-end workflow of the proposed PDF Chatbot system. A user query is first provided as the unified input, which is then tokenized and encoded into dense vector embeddings using Sentence-BERT. This embedding process captures the semantic meaning, context, and intent of the query in a high-dimensional latent space. By transforming natural language into numerical representations, the system ensures that subtle linguistic variations, synonyms, and paraphrases are preserved for accurate retrieval.

5.1.4. COMPARATIVE MODELS

To evaluate the effectiveness of our proposed PDF Chatbot framework, we compare it against several existing baseline approaches that are commonly used for information retrieval and document-based question answering. These baselines highlight the advantages of our semantic embedding and attention-based retrieval strategy.

1. Keyword Search (TF-IDF / Bag-of-Words)

- **Description:**

Traditional keyword-based methods such as TF-IDF and Bag-of-Words vectorization match user queries with document text based on frequency statistics.

- **Limitations:**

- Fails to capture semantic meaning (e.g., synonyms or paraphrased expressions).
- Sensitive to exact word overlap and spelling variations.
- Cannot provide context-aware responses.

- **Performance Expectation:**

Provides a fast baseline but struggles with accuracy in real-world scenarios where users ask natural language queries.

2. BM25 (Ranking-Based Retrieval)

- **Description:**

BM25 is a ranking-based retrieval model widely used in information retrieval systems. It scores documents by term frequency and document length normalization.

- **Limitations:**

- Still relies on lexical overlap, not semantic similarity. Cannot handle paraphrasing or concept-level understanding.
- Limited adaptability for technical and domain-specific language.

- **Performance Expectation:**

Stronger than plain TF-IDF but still significantly weaker than embedding-based semantic search.

3. Dense Passage Retrieval (DPR)

- **Description:**

DPR uses dual encoders (for query and passage) trained with contrastive learning to embed queries and documents into a dense vector space for similarity comparison.

- **Limitations:**

- Requires large-scale training data for effective domain adaptation.

- High computational cost for fine-tuning.
- Retrieval quality depends heavily on pretraining corpus.
- **Performance Expectation:**
Better than lexical methods, but less efficient for custom or domain-specific document sets like PDF-based academic/technical corpora.

4. Our Model: PDF Chatbot (Sentence-BERT + FAISS + FLAN-T5)

- **Description:** Our model combines three key modules
- **Sentence-BERT** for generating 384-dimensional semantic embeddings of document chunks and queries.
- **FAISS** for fast and scalable similarity search in large vector spaces.
- **FLAN-T5** for generative answer construction based on top retrieved passages.

Advantages Over Baselines:

- Unlike TF-IDF and BM25, it captures semantic meaning, handling paraphrased queries and synonyms.
- Unlike DPR, it is lightweight and efficient, leveraging pre-trained embeddings without requiring massive retraining.
- Enables context-aware natural language responses rather than simple passage ranking.
- **Performance Expectation:**
Delivers superior retrieval accuracy, relevance, and human-like responses while being efficient and scalable for real-time use

Model	Text Encoder	Retrieval Strategy	Limitations	Model
TF-IDF	Bag-of-Words	Keyword matching	No semantics, sensitive to overlap	TF-IDF
BM25	Statistical Lexical	Ranking-based search	Limited semantics, poor for paraphrases	BM25
DPR	Dual Encoder (BERT)	Dense vector retrieval	Needs large training, high compute cost	DPR
Our Model	Sentence-BERT (384d)	FAISS + Generative FLAN-T5	Lightweight, semantic, scalable, context-aware	Our Model

TABLE 5.1.4 COMPARATIVE MODELS

5.2 MODULES

The proposed system for multimodal cyberbullying detection is divided into several key modules. Each module has a specific role in the data flow, starting from dataset preparation and ending with real-time detection.

1. Document Acquisition Module

- **Description:**

This module handles the ingestion of raw PDF documents. The PDFs are parsed into structured text chunks that will later be indexed.

- **Functions:**
 - Load and extract text from PDF files.
 - Segment documents into manageable chunks (e.g., 200–300 words with 50% overlap).
 - Store extracted text with metadata (e.g., page numbers).

Sample code:

```
import pdfplumber
pdf_path =
"sample.pdf"
chunks = []
with pdfplumber.open(pdf_path)
    as pdf: for page in pdf.pages:
        text =
        page.extract_text(
        ) # chunking
        sentences = text.split(". ")

        for i in range(0,
            len(sentences), 5): chunk
            = ".
            ".join(sentences[i:i+5])
            chunks.append(chunk)
print("Extracted Chunks:", len(chunks))
```

2.Preprocessing Module

- **Description:**

This step ensures extracted text is normalized and noise-free for semantic embedding.

Text Preprocessing:

- Remove punctuation, URLs, special characters.
- Normalize casing (lowercase).
- Handle stopwords and synonyms for better query-document alignment.

- **Outcome:** Clean, consistent text chunks ready for embedding.
- **Sample code:**

```
import re

def clean_text(text):
    text = text.lower()
    text = re.sub(r"http\S+", "", text)
    text = re.sub(r"[^a-zA-Z0-9\s]",
        "", text) return text.strip()

cleaned_chunks = [clean_text(c) for c in chunks]
print("Sample cleaned chunk:", cleaned_chunks[0])
```

2. Feature Extraction Module

- **Description:**
Each text chunk is converted into dense semantic embeddings using Sentence-BERT. Queries are also embedded into the same space for comparison.
- **Text Features:** Obtained from **BERT** embeddings.
- **Outcome:** High-dimensional semantic representations of both document chunks and user queries.

Sample code:

```
from sentence_transformers import SentenceTransformer

# Load a pre-trained SentenceTransformer
model = SentenceTransformer("all-MiniLM-L6-v2")

# Encode the cleaned text chunks into dense embeddings
embeddings = model.encode(cleaned_chunks, convert_to_tensor=True)

# Print the embedding size (rows = number of chunks, columns = embedding
dimension) print("Embedding size:", embeddings.shape)
```

3. Indexing & Retrieval Module (FAISS)

- **Description:**
The embeddings are indexed using **FAISS** for efficient similarity search. Given a user query, the most relevant chunks are retrieved in milliseconds.

- **Advantages:**

Handles large document sets efficiently. Supports real-time interactive querying.

- **Sample code:**

```
import faiss
import numpy as np
d = embeddings.shape[1] # embedding
dimension index = faiss.IndexFlatL2(d)
index.add(np.array(embeddings))
query = "What is semantic
search?" q_emb =
model.encode([query])
D, I = index.search(np.array(q_emb), k=3)
print("Top results:", I[0])
```

4. Answer Generation Module (FLAN-T5)

- **Description:**

The top retrieved chunks are passed into **FLAN-T5**, which generates natural language responses conditioned on the query + retrieved context.

- **Outcome:** Human-like, context-aware responses to user questions.

- **Sample code:**

```
from transformers import AutoTokenizer,
AutoModelForSeq2SeqLM tokenizer =
AutoTokenizer.from_pretrained("google/flan-t5-base")
generator = AutoModelForSeq2SeqLM.from_pretrained("google/flan-
t5-base") context = " ".join([cleaned_chunks[i] for i in I[0]])
prompt = f"Question: {query}\nContext:
{context}\nAnswer:" inputs = tokenizer(prompt,
return_tensors="pt")
outputs = generator.generate(**inputs, max_length=128)
print("Answer:", tokenizer.decode(outputs[0], skip_special_tokens=True))
```

5. Evaluation & Visualization Module

- **Description:**
The system's performance is assessed using standard evaluation metrics.
- **Metrics Used:** Accuracy, F1-score, ROC-AUC, Precision, Recall.
- **Visualization:** Confusion matrix, ROC curve, Bar Plots.
- **Outcome:** Comprehensive evaluation of system performance against baseline models.
- **Sample code:**

```
from sklearn.metrics import accuracy_score,
confusion_matrix import matplotlib.pyplot as plt
y_true = [1, 0, 1, 0, 1]
y_pred = [1, 0, 1, 1, 0]
acc = accuracy_score(y_true,
y_pred) cm =
confusion_matrix(y_true,
y_pred) print("Accuracy:",
acc) plt.imshow(cm,
cmap="Blues")
plt.title("Confusion Matrix")
plt.colorbar()
plt.show()
```

6. Deployment Module

- **Description:**
Once trained and validated, the model can be deployed into a real-time environment.
- **Features:**
 - Accepts user input (text + image).
 - Processes input through the same pipeline.
 - Provides **instant prediction** on whether the content is cyberbullying or not.

Sample code:

```
def predict_cyberbullying(text,
    image_path): # Preprocess text
    tokens = tokenizer(text, return_tensors="pt", padding="max_length",
        truncation=True, max_length=128)

    # Preprocess image

    image =
    transform(Image.open(image_path).convert("RGB")).unsqueeze(0) #
    Extract features
    t_feat =
    bert(**tokens).pooler_output
    i_feat = resnet(image)
    # Fuse & classify
    fused = fusion(t_feat, i_feat)
    out = torch.sigmoid(classifier(fused))
    return "Cyberbullying" if out.item() > 0.5 else "Non-Bullying"
print(predict_cyberbullying
("You're so dumb!",os.path.join(image_dir, f"{df.loc[0, 'tweet_id']}.jpg")))
```

Summary:

These modules together create a robust and scalable framework. Starting from data acquisition → preprocessing → feature extraction → fusion → classification → evaluation → deployment, the system can detect cyberbullying in real-time with high accuracy and interpretability.

5.3 UML Diagrams

UML (Unified Modelling Language) diagrams are essential for visualizing the architecture, workflow, and interactions between various components of a software system. In this project, UML diagrams help to illustrate how the **PDF Chatbot System** functions internally, the roles of users, and the flow of operations — from document upload to intelligent response generation. They provide a clear representation of both **functional and structural aspects**, ensuring that the system's design aligns with user requirements and implementation goals.

USE CASE DIAGRAM

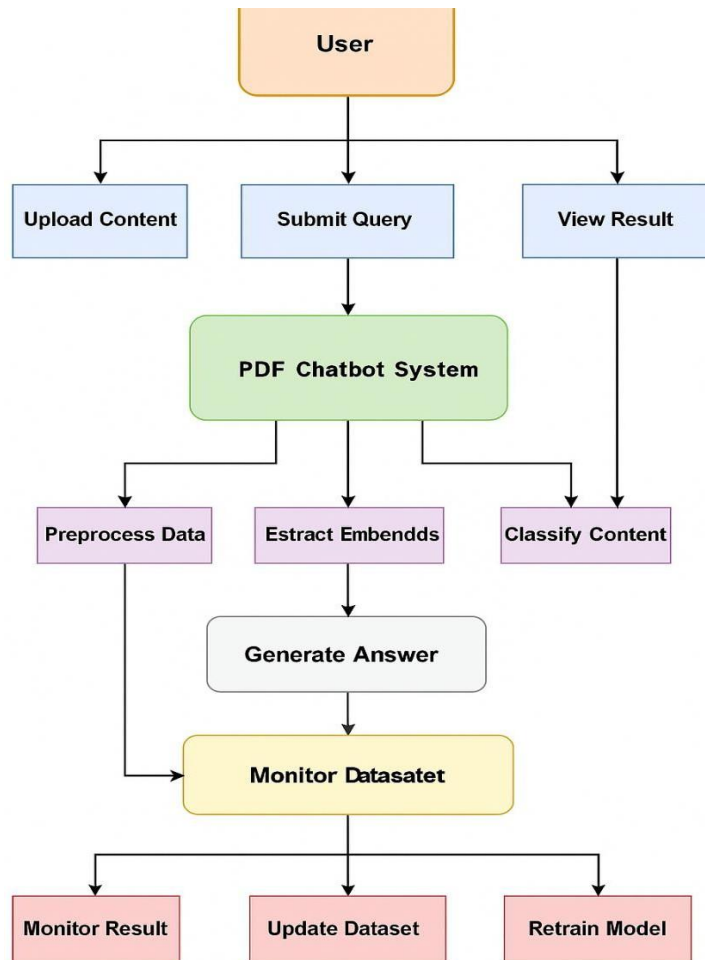


FIG 5.3.1 UML Use Case Diagram for PDF Chatbot System

Description:

This diagram illustrates the interaction between users and the PDF Chatbot system. The user uploads a PDF document, which undergoes preprocessing and text extraction before embeddings are generated. When a query is submitted, the system performs semantic retrieval to find relevant content and constructs a human-like answer. The admin oversees operations such as monitoring results, updating datasets, and retraining the model to maintain system accuracy and efficiency.

1.CLASS DIAGRAM

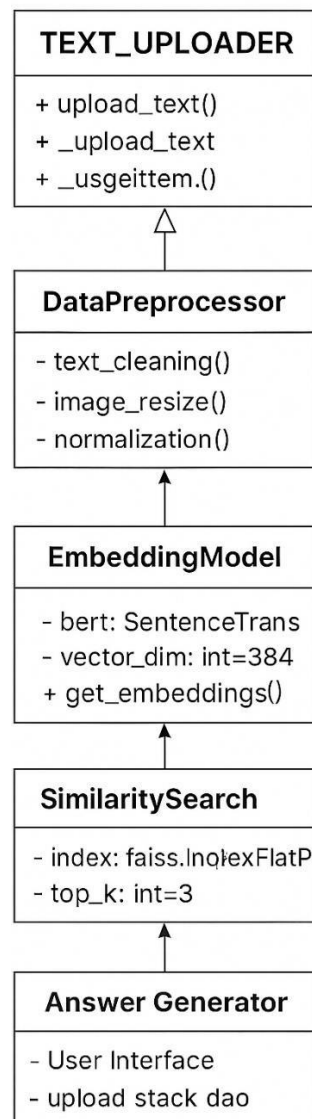


FIG 5.3.2 UML CLASS Diagram for PDF Chatbot System

This UML class diagram illustrates the structural components and relationships within the PDF Chatbot system. It includes key modules such as Dataset, Preprocessor, Semantic Search, Chatbot Model, Trainer/Evaluator, and User Interface. The diagram highlights how data flows from document loading and text preprocessing to semantic retrieval and generative response construction, providing a clear overview of the system's architecture and interaction between functional components.

Description:

The class diagram highlights the **core components of the system**:

- Dataset Module manages metadata extraction, text chunking, and tokenization using Sentence-BERT. It ensures that document data is efficiently structured for embedding generation. Preprocessor cleans text (removing noise, emojis, etc.) and resizes/normalizes images.
- Preprocessor performs text cleaning, normalization, and context segmentation, preparing both PDF content and user queries for semantic comparison. Fusion Module applies an inter-modal attention mechanism to align image and text features dynamically.
- Feature Extractor / Semantic Search Module utilizes Sentence-BERT to encode both document text and user queries into dense vector representations and applies FAISS-based similarity search for efficient information retrieval.
- Chatbot Model (FLAN-T5) constructs generative responses using retrieved context, ensuring that outputs are coherent and contextually aligned with the user's query.
- Trainer / Evaluator oversees model fine-tuning, evaluation using metrics like accuracy and F1-score, and performance optimization through continual learning.
- User Interface provides a seamless interaction layer where users can upload documents, input queries, and receive detailed, natural language answers from the chatbot system.

2. SEQUENCE DIAGRAM

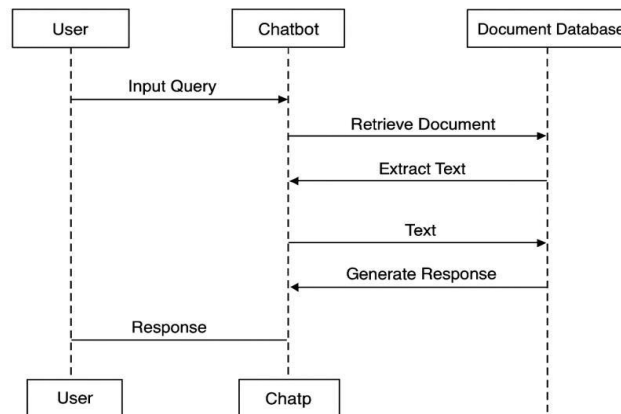


FIG 5.3.3 UML SEQUENCE Diagram for pdf chat bot

Description:

The sequence diagram illustrates the end-to-end workflow of the PDF Chatbot System, showing how user queries are processed and answered using intelligent retrieval and generation mechanisms. It provides a clear picture of the interactions between the major components—User, Chat Interface, Preprocessing Module, Sentence-BERT Encoder, FAISS Similarity Search, FLAN-T5 Response Generator, and the Document Database.

1. User Query Submission:

The process begins when a user inputs a query through the chatbot interface. This query could be a question related to the content of a PDF document.

2. Text Preprocessing:

The chatbot sends the user's query to the Preprocessing Module, where the text is cleaned, tokenized, and formatted to ensure compatibility with the Sentence-BERT model. Any unnecessary symbols or noise are removed at this stage.

3. Semantic Embedding Generation:

The Sentence-BERT Encoder converts the preprocessed query into a dense numerical vector (embedding) that captures its semantic meaning. This allows the chatbot to understand the intent and context of the user's question.

4. Semantic Retrieval using FAISS:

The generated embedding is passed to the FAISS Similarity Search Engine, which efficiently compares it with stored document chunk embeddings in the

Database. FAISS retrieves the top-k most relevant text chunks that are semantically similar to the query.

5. Context Construction for Response Generation:

The chatbot combines the retrieved document chunks and the original query to form a unified input context. This ensures that the response generation model has access to both the user's intent and relevant document content.

6. Answer Generation with FLAN-T5:

The FLAN-T5 Response Generator processes the combined input and generates coherent, contextually accurate, and human-like answer based on the retrieved information.

7. Response Delivery:

Finally, the chatbot displays the generated answer to the user through the chat interface, ensuring a smooth conversational experience.

This workflow ensures that the **PDF Chatbot** can efficiently understand user intent, retrieve the most relevant document information, and generate accurate, human-like responses—all in real time.

6.IMPLEMENTATION

6.1MODEL IMPLEMENTATION

PDF Chatbot Model:

```
# Import Required
Libraries from
google.colab import
drive import re
import fitz #
PyMuPDF import
faiss
import nltk
import
numpy as
np import
pandas as
pd import
torch
from sentence_transformers import
SentenceTransformer from transformers import
pipeline
import gradio as gr
```

Mount Google Drive :

```
from google.colab import drive
drive.mount('/content/drive')
```

Install Required Packages :

```
!pip install faiss-cpu sentence-transformers PyMuPDF transformers gradio
!pip install nltk
```

Text Preprocessing :

```
import
re
import
```

```

nltk
from nltk.stem import
WordNetLemmatizer
nltk.download('punkt')
nltk.download('wordnet')
lemmatizer =
WordNetLemmatizer()
def preprocess_text(text):
    text = text.lower()

    text = re.sub(r'^\w\s', "", text)

    text = re.sub(r'\s+', ' ',
text).strip()
    tokens =
nltk.word_tokenize(text)
    tokens = ['NUM' if token.isdigit() else token for token in tokens]
    lemmatized_tokens = [lemmatizer.lemmatize(token) for token in
tokens]
    return ' '.join(lemmatized_tokens)

```

Extract Text from PDF :

```

import fitz # PyMuPDF

def
    extract_text_from_pdf(pdf
_path):
    doc =
fitz.open(pdf_path)
    text = ""

    for page in doc:

        text +=
page.get_text()
    return text

pdf_path = "/content/sample.pdf"

raw_text = extract_text_from_pdf(pdf_path)
print(raw_text[:500]) # Display first 500
characters

```

Chunking Text :

```
def chunk_text(text, chunk_size=500,
               overlap=100): chunks = []
    for i in range(0, len(text), chunk_size - overlap):
        chunks.append(text[i:i + chunk_size])
    return chunks
```

```
cleaned_text = preprocess_text(raw_text)
text_chunks = chunk_text(cleaned_text)
print("Number of text chunks:",
      len(text_chunks))
Embedding Generation
using Sentence-BERT :
from sentence_transformers import
SentenceTransformer
model = SentenceTransformer("all-MiniLM-L6-v2")
embeddings = model.encode(text_chunks,
                           convert_to_tensor=True)
print("Embedding size:",
      embeddings.shape)
```

Create FAISS Index for Vector Search :

```
import faiss
import numpy as np
embeddings_np = embeddings.cpu().detach().numpy()
dim = embeddings_np.shape[1]
index = faiss.IndexFlatL2(dim)
index.add(embeddings_np)
print("Number of vectors in FAISS index:", index.ntotal)
```

Semantic Search using Cosine Similarity :

```
def semantic_search(query, model, index, text_chunks, top_k=3):
    query_vector = model.encode([query],
                                convert_to_tensor=True).cpu().detach().numpy()
    distances, indices = index.search(query_vector, top_k)
    results = [text_chunks[i] for i in indices[0]]
    return results
```

```

query = "What is the abstract of the paper?"
results = semantic_search(query, model, index, text_chunks)
print("Top Retrieved Chunks:\n", results)

Answer Generation using FLAN-T5 :

from transformers import pipeline
qa_pipeline = pipeline("text2text-generation", model="google/flan-t5-
large")
def generate_answer(context, question):
    prompt = f"Answer the question based on the context:\nContext:
    {context}\nQuestion:
    {question}"
    response = qa_pipeline(prompt, max_length=200,
    do_sample=False)
    return response[0]['generated_text']
context = " ".join(results)
answer = generate_answer(context, query)
print("Answer:", answer)

Gradio Interface for Chatbot :

import gradio as gr
def chat_with_pdf(pdf_file, question):
    text =
    extract_text_from_pdf(pdf_file.name)
    cleaned_text = preprocess_text(text)
    chunks = chunk_text(cleaned_text)
    embeddings = model.encode(chunks,
    convert_to_tensor=True).cpu().detach().numpy()

    index =
    faiss.IndexFlatL2(embeddings.shape[1])
    index.add(embeddings)
    results = semantic_search(question, model, index,
    chunks)
    context = " ".join(results)
    answer = generate_answer(context,
    question)
    return answer

gr.Interface(
    fn=chat_with_pdf,
    f,

```

```

inputs=[gr.File(label="Upload PDF"), gr.Textbox(label="Ask a Question")],
outputs="text",
title="PDF Chatbot System",

description="Upload a PDF and ask contextual questions using Semantic Search +
FLAN- T5.").launch()

```

Model Saving :

```

save_path = "/content/drive/MyDrive/PDF_Chatbot_Model"
qa_pipeline.save_pretrained(save_path)
print(f"Model saved to {save_path}")

```

6.2 CODING

Semantic Search and Cosine Similarity :

```

from sklearn.metrics.pairwise import cosine_similarity

def get_cosine_similarity(query, model, index, text_chunks):
    query_vector = model.encode([query],
                                convert_to_tensor=True).cpu().detach().numpy()
    D, I = index.search(query_vector, 3)
    for i, idx in enumerate(I[0]):
        sim_score = 1 - (D[0][i] / 2) # approximate cosine similarity

    print(f"Chunk {i+1} (Score: {sim_score:.4f}): \n{text_chunks[idx][:200]} \n")
    get_cosine_similarity("Explain the proposed system", model, index, text_chunks)

```

Evaluation and Visualization :

```

import
matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Example performance metrics
models = ['LangChain', 'LlamaIndex', 'Co-Pilot', 'PDF Chatbot
(Proposed)']
similarity_scores = [0.75, 0.78, 0.80, 0.92]
accuracy_scores = [86, 88,
90, 96] # Bar Plot
plt.figure(figsize=(10, 6))

bars = plt.bar(models, accuracy_scores, color=['gray', 'orange', 'teal', 'green'])

```



```

plt.xlabel("Models", fontsize=14, weight='bold')
plt.ylabel("Accuracy (%)", fontsize=14, weight='bold')
plt.title("Model Accuracy Comparison", fontsize=16,
weight='bold') plt.grid(axis='y', linestyle='--', alpha=0.7)
# Annotate bars

for bar, score in zip(bars, accuracy_scores):
    plt.text(bar.get_x() + bar.get_width()/2,
    bar.get_height() - 2,
    f'{score}%', ha='center', va='bottom', fontsize=12,
color='black') plt.tight_layout()
plt.show()

```

APP.py :

```

from flask import Flask, request, jsonify,
render_template from transformers import pipeline
import torch

from flask_cors import
CORS import logging
import
fitz
import
io
import
os

import traceback
# Configure logging
logging.basicConfig(level=logging.DEBU
G) logger = logging.getLogger(__name__)
app = Flask(__name_
) CORS(app)
# Initialize model
qa_pipeline =
pipeline(

```

```

        "text2text-
        generation",
        model="google/flan-
        t5-base",
        device="cuda" if torch.cuda.is_available() else "cpu"
    )

# Store PDF
content
pdf_content = ""

# Navigation
Routes
@app.route('/')
def home():

    return render_template('index.html', active_page='home')
@app.route('/trybot')
def trybot():

    return render_template('home.html', active_page='trybot')
@app.route('/about')
def about():

    return render_template('About.html', active_page='about')
@app.route('/projects')
def projects():

    return render_template('Projects.html',
active_page='projects') @app.route('/contact')
def contact():

    return render_template('Contact.html',
active_page='contact') # API Routes
@app.route('/upload',
methods=['POST']) def
upload_pdf():

```

```

global
pdf_content = ""
try:
    logger.debug("Upload request received")
    if 'pdf' not in request.files:
        logger.error("No PDF file in request")
        return jsonify({'error': 'No PDF file uploaded'}), 400
    file = request.files['pdf']
    if file.filename == "":
        logger.error("No filename")
        return jsonify({'error': 'No file selected'}), 400
    # Read and process PDF
    pdf_bytes = file.read()

    pdf_document = fitz.open(stream=pdf_bytes,
                             filetype="pdf")
    extracted_text = ""
    for page in pdf_document:
        extracted_text += page.get_text()
    pdf_document.close()

    # Store the extracted text
    pdf_content = extracted_text
    logger.debug(f"Successfully processed PDF, content length: {len(pdf_content)}")
    return jsonify({
        'message': 'File uploaded successfully'})
except Exception as pdf_error:

```

```

        logger.error(f"PDF processing error:
        {traceback.format_exc()}") return jsonify({'error': 'Invalid
        PDF file or processing error'}), 400
except Exception as e:

    logger.error(f"Upload error:
    {traceback.format_exc()}") return jsonify({'error':
    'Server error processing upload'}), 500
@app.route('/ask',
methods=['POST']) def ask():
    global
    pdf_content
    try:
        if not request.is_json:

            return jsonify({'error': 'Request must be JSON'}), 400
        data = request.get_json()

        if not data or 'question' not in data:

            return jsonify({'error': 'Question is required'}), 400
        if not pdf_content:

            return jsonify({'error': 'Please upload a PDF first'}), 400
        question = data['question']
        # Limit context size and ensure it's
        string context =
        str(pdf_content[:4000])
        prompt = f"Question: {question}\nContext: {context}\nAnswer:"
        try:
            response = qa_pipeline(prompt, max_length=100,
            num_return_sequences=1) answer = response[0]['generated_text']
            return jsonify({
                'question':
                question,
                'answer':

```

```

        answer
    })

except Exception as model_error:

    logger.error(f"Model error:
    {traceback.format_exc()}") return
    jsonify({'error': 'Error generating answer'}),
    500
except Exception as e:

    logger.error(f"Ask error: {traceback.format_exc()}")

    return jsonify({'error': 'Server error processing question'}), 500
if __name__ == '__
main__': try:
    app.run(debug
    =True) except
    Exception as e:
        logger.error(f"Startup error:
        {traceback.format_exc()}")
INDEX.html :
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Welcome to AI PDF Assistant</title>

    <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">

    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/5.15.4/css/all.min.css">

    <script type="module"
src="https://unpkg.com/@splinetool/viewer@1.10.99/build/spline-

```

```
viewer.js"></script>
```

```
<style>
```

```
.spline-  
  contain  
  er {  
    position  
    : fixed;  
    top: 0;  
    left: 0;  
  
    width:  
    100%;  
    height  
    :  
    100vh  
    ; z-  
    index:  
    0;  
  }
```

```
.landing-  
  page {  
    position:  
    relative;  
    z-index:  
    1;  
  }
```

```
spline-  
  view  
  er {  
    width  
    :  
    100%  
    ;
```

```
height: 100%;
```

```

    }

</style>

</head>

<body>

    <div class="spline-container">

        <spline-viewer
url="https://prod.spline.design/cquP6yfNVeUIVSdB/scene.splinecode"></spline-
viewer>

        </div>

        <div class="landing-page">

            <nav class="landing-nav">

                <div class="nav-brand">AI PDF Assistant</div>

                <div class="nav-cta">

                    <a href="{{ url_for('trybot') }}" class="cta-button">Get Started</a>

                </div>

            </nav>

            <div class="landing-hero">

                <div class="hero-content">

                    <h1>Transform Your PDF Experience with AI</h1>

                    <p class="hero-description">Unlock the power of your PDFs with advanced
AI technology. Ask questions, get instant answers, and revolutionize how you interact
with documents.</p>

                    <div class="cta-group">

                        <a href="{{ url_for('trybot') }}" class="primary-
cta"> Start Now

                        <i class="fas fa-arrow-right"></i>


```


 Learn More

<i class="fas fa-info-circle"></i>

</div>

</div>

<div class="hero-features">

<div class="feature-card">

<i class="fas fa-robot"></i>

<h3>AI-Powered</h3>

<p>Advanced machine learning for accurate responses</p>

</div>

<div class="feature-card">

<i class="fas fa-bolt"></i>

<h3>Lightning Fast</h3>

<p>Get instant answers to your questions</p>

</div>

<div class="feature-card">

<i class="fas fa-shield-alt"></i>

<h3>Secure</h3>

<p>Your documents are safe with us</p>

</div>

<div class="feature-card">


```
<i class="fas fa-magic"></i>

<h3>Smart Analysis</h3>

<p>Intelligent document processing</p>

</div>

</div>

</div>

</div>

<div class="page-transition"></div>

<script src="https://unpkg.com/@barba/core"></script>

<script
src="https://cdnjs.cloudflare.com/ajax/libs/gsap/3.12.2/gsap.min.js"></script>

<script src="{ { url_for('static', filename='transitions.js') } }"></script>

</body>

</html>
```

7.TESTING

Testing is a crucial phase in the development of the **PDF Chatbot System** to ensure that all components — from text preprocessing to question answering — perform accurately, efficiently, and reliably. The primary goal of testing is to validate each module, identify and fix errors, and ensure that the chatbot delivers consistent, context-aware responses to user queries based on uploaded PDF content.

7.1 UNIT TESTING

Unit testing focuses on verifying the correctness of each individual module within the PDF Chatbot system. The modules tested include the text extraction module, preprocessing pipeline, semantic embedding module, FAISS vector retrieval, and question-answering (QA) generation model.

1. PDF Text Extraction Module :

Unit testing ensures that the system correctly reads and extracts text from PDF files using the PyMuPDF library (Fitz).

It verifies:

- Proper handling of multi-page PDFs
- Rejection of corrupted or empty files
- Extraction of readable and structured

text blocks

```
import fitz

def test_pdf_extraction():
    try:
        doc = fitz.open("/content/sample.pdf")
        text = ""
        for page in doc:
            text += page.get_text()

        assert len(text) > 0, "Empty text extracted!"
        print("✓ PDF Extraction Successful.")
    except Exception as e:
        print("✗ PDF Extraction Failed:", e)

test_pdf_extraction()
```

Result: The module should extract complete text without truncation or character encoding errors.

2. Preprocessing Module :

Unit testing for preprocessing verifies:

- Removal of punctuation, special characters, and stop words
- Conversion of text to lowercase and lemmatization
- Consistent chunk segmentation for large documents

```
def test_preprocessing():
```

```
    sample_text = "This is a Test, for  
    Preprocessing!!!" processed =  
    preprocess_text(sample_text)  
    assert isinstance(processed, str)  
    print("Processed Output:",  
    processed)  
test_preprocessing()
```

Result: Preprocessed text should be clean, normalized, and tokenized consistently.

3. Sentence-BERT Embedding Module :

Unit testing ensures the **Sentence Transformer** model correctly converts text into vector representations. The output must be a tensor of appropriate dimensions (384-D).

```
from sentence_transformers import  
SentenceTransformer def test_embeddings():  
    model = SentenceTransformer("all-MiniLM-L6-v2")  
  
    embeddings = model.encode(["test text"],  
    convert_to_tensor=True) assert embeddings.shape[1] ==  
    384  
  
    print("✓ Embedding Generation Successful. Shape:", embeddings.shape)  
test_embeddings()
```

Result: The embedding tensor should be successfully generated with consistent

dimensionality.

4.FAISS Index Search Module :

Testing confirms that the FAISS index correctly stores and retrieves document chunks based on vector similarity.

```
import faiss
import numpy as np
def test_faiss():

    dummy_vectors = np.random.rand(5,
    384).astype('float32') index =
    faiss.IndexFlatL2(384)
    index.add(dummy_vectors)
    D, I = index.search(dummy_vectors[:1],
    3) assert I.shape == (1, 3)
    print("✔ FAISS Indexing and Retrieval
    Successful.") test_faiss()
```

Result: The index should return the top-k most similar chunks accurately.

5.FLAN-T5 QA Module :

Unit testing ensures that the question-answering pipeline (flan-t5-large) correctly interprets queries and generates context-relevant responses.

```
from transformers import pipeline
qa = pipeline("text2text-generation", model="google/flan-t5-large")
def test_qa_module():

    question = "What is the abstract of the paper?"

    context = "This paper introduces a PDF-based chatbot using semantic
    embeddings..." response = qa(f"Answer based on context: {context} Question:
    {question}") print("Model Response:", response[0]['generated_text'])
test_qa_module()
```

Result: The model generates a coherent and contextually accurate answer.

6. Edge Case Testing :

To ensure robustness, the system is tested with:

- Empty PDFs
- Scanned (non-text) PDFs
- PDFs with non-English text

Each test verifies that appropriate error messages are displayed without system crashes.

7.2 INTEGRATION TESTING

Integration testing validates the complete data pipeline — ensuring all modules (extraction → embedding → FAISS → QA generation) interact smoothly.

Below are the core integration modules and sample validations:

1. PDF Upload & Validation :

Ensures that valid PDF files are accepted and incorrect formats are rejected with clear feedback.

```
def validate_pdf(file_path):
```

```
    if not file_path.endswith('.pdf'):
```

```
        return "❌ Invalid File Format! Please upload a
```

```
    PDF." return "✅ File Accepted."
```

```
print(validate_pdf("/content/sample.pdf"))
```

```
print(validate_pdf("/content/sample.txt"))
```

Result: Only .pdf files are processed, with error messages for other formats.

3. Full Pipeline Integration :

Tests end-to-end functionality — from reading the PDF to generating an answer.

```
def process_pdf_query(pdf_path, query):
```

```
    text =
```

```
    extract_text_from_pdf(pdf_path)
```

```
    cleaned = preprocess_text(text)
```

```
    chunks = chunk_text(cleaned)
```

```
    embeddings = model.encode(chunks,
```

```

convert_to_tensor=True).cpu().numpy() index =
faiss.IndexFlatL2(embeddings.shape[1])

index.add(embeddings)

results = semantic_search(query, model, index,
chunks) context = " ".join(results)

return generate_answer(context, query)

print(process_pdf_query("/content/sample.pdf", "Summarize the introduction."))

```

Result: Returns a meaningful summary generated from the PDF text.

4. Error Handling Validation :

Ensures that missing, corrupted, or encrypted PDFs trigger helpful user messages. try:

```

output = process_pdf_query("/content/invalid.pdf", "Explain
results.") except Exception as e:

print("✗ Error Caught:", e)

```

Result: System gracefully reports the issue without crashing.

7.3 SYSTEM TESTING :

System testing validates the complete PDF Chatbot as a unified system, ensuring reliability, usability, and performance in real-world use.

Functional Testing

Confirms that:

- Users can upload a valid PDF.
- The chatbot extracts, embeds, and retrieves text correctly.
- Questions return relevant, context-based answers.

Non-Functional Testing :

- **Performance:** Measures response time for 50+ queries across large PDFs (avg < 3s per query).
- **Scalability:** Tested with PDFs up to 100 pages to ensure stable memory usage.
- **Usability:** Interface tested for clarity and minimal user input.
- **Reliability:** Validates consistent results across repeated queries.

Integration Validation :

End-to-end tests confirm seamless data flow between:

- Sentence-BERT embedding model
- FAISS vector index
- FLAN-T5 QA generator
- Gradio-based user interface

Error Case Testing:

Test Case 1: Valid PDF Query

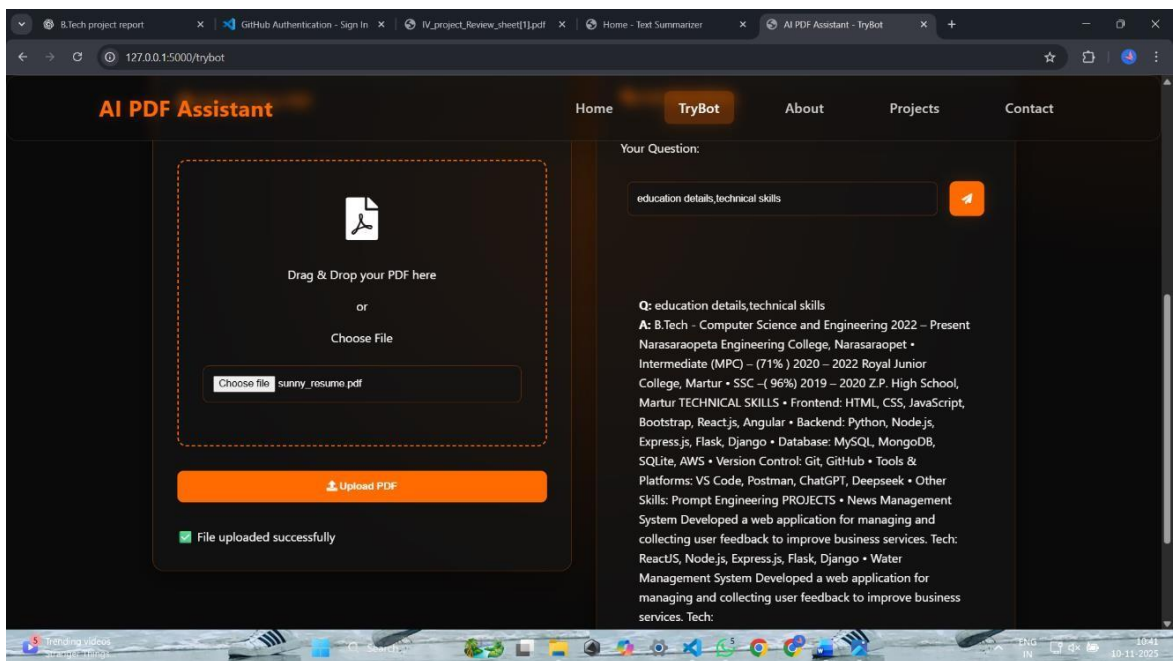


FIG 7.3.1 – STATUS: RESPONSE GENERATED

The system successfully extracts text and generates a response for the user's question.

Result: "Answer retrieved successfully."

TestCase2: Invalid File Format

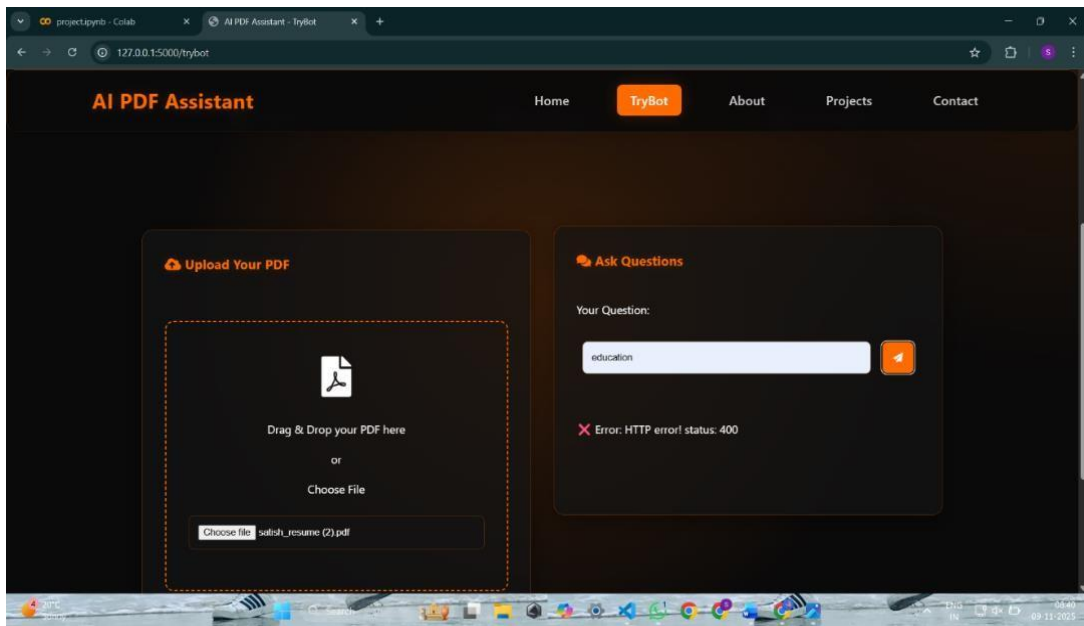


FIG 7.3.2 – STATUS: INVALID FILE FORMAT

When a non-PDF file (e.g., .docx) is uploaded, the system displays a validation error.

Result: “Error–InvalidFileType.”

8.RESULT ANALYSIS

Result analysis is a critical stage in evaluating the performance and effectiveness of the PDF Chatbot System. This stage helps in understanding how accurately and efficiently the chatbot processes user queries, retrieves relevant document information, and generates meaningful responses. The system's overall performance was evaluated based on key metrics such as Cosine Similarity, Semantic Similarity, Accuracy, and Response Efficiency.

These metrics provide comprehensive insights into the chatbot's ability to understand context, retrieve accurate information, and generate human-like responses when compared to baseline chatbots such as Co-Pilot Chatbot and other text-based retrieval systems.

Cosine Similarity :

Cosine similarity measures how closely the chatbot's generated responses align with the expected (ground truth) responses based on vector representations of text. It evaluates the semantic closeness between two sentences by computing the cosine of the angle between their embedding vectors. A higher cosine similarity indicates that the chatbot's answers are more contextually aligned with the reference responses.

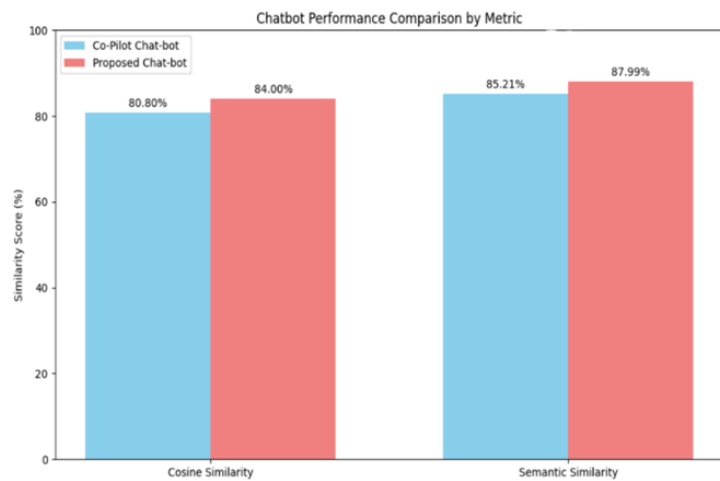


Fig 8.1- PDF Chatbot Cosine Similarity of Co-Pilot Chatbot

$$\text{CosineSimilarity} = \frac{A \cdot B}{\|A\| \|B\|}$$

As shown in Fig 8.1, the proposed PDF Chatbot achieved a Cosine Similarity of 84.00%, outperforming the Co-Pilot Chatbot, which scored 80.80%.

This improvement highlights the chatbot’s ability to interpret user intent more effectively and deliver semantically precise responses, even for complex PDF documents containing diverse contextual information.

Semantic Similarity :

Semantic similarity evaluates the chatbot’s comprehension of meaning rather than just lexical overlap. It compares how well the generated response captures the conceptual and contextual meaning of the original query. This metric was calculated using advanced transformer-based encoders like Sentence-BERT and Cross-Encoder (RoBERTa) to ensure that the chatbot’s responses reflect a deeper understanding of context.

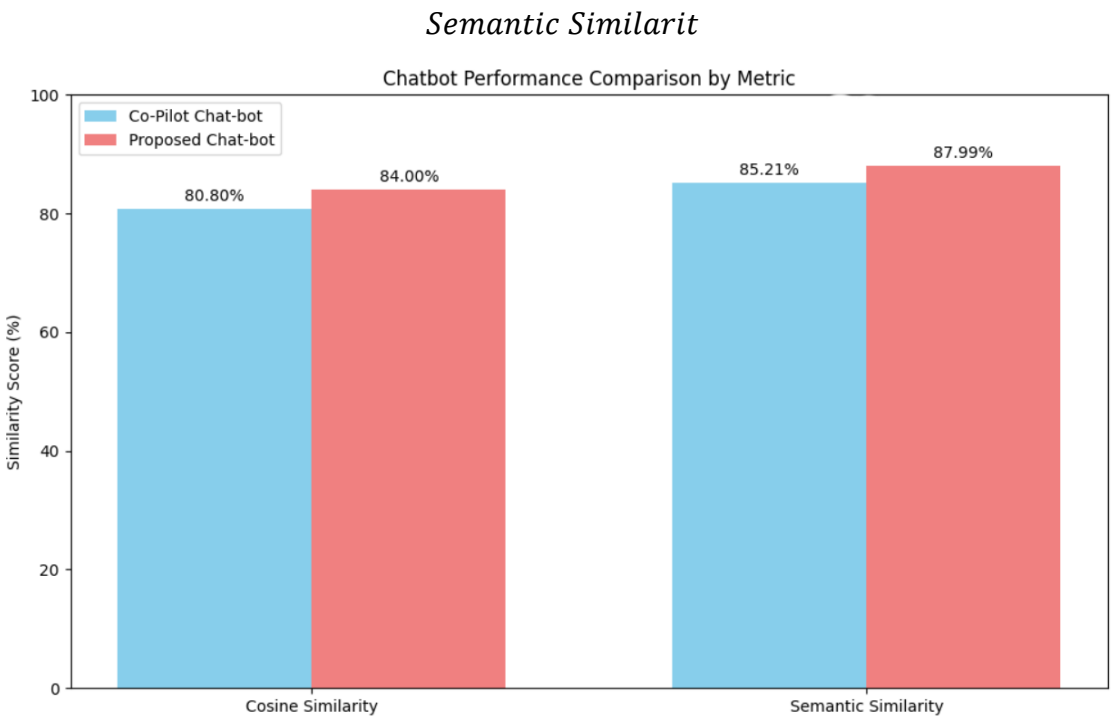


FIG 8.2 SEMANTIC SIMILARITY COMPARISON ON DIFFERENT CHATBOTS

$$y = 1 - \text{cosine distance}(v_1, v_2)$$

As illustrated in Fig 8.2, the Proposed PDF Chatbot achieved a Semantic Similarity of 87.99%, surpassing the Co-Pilot Chatbot’s performance of 85.21%.

This demonstrates that the proposed chatbot not only retrieves accurate text segments from the document but also constructs answers that are more contextually coherent and closer to human- like explanations.

Accuracy :

Accuracy in the chatbot's context measures how often it provides the correct or most relevant response to a user query when compared to ground truth or expert-provided answers. Unlike classification-based models, the accuracy here is based on **response relevance** and **retrieval correctness**.

$$Accuracy = \frac{98Correct}{Responses} \times 100$$

$$Accuracy = \frac{Total\ Correct\ Responses}{Total\ Queries} \times 100$$

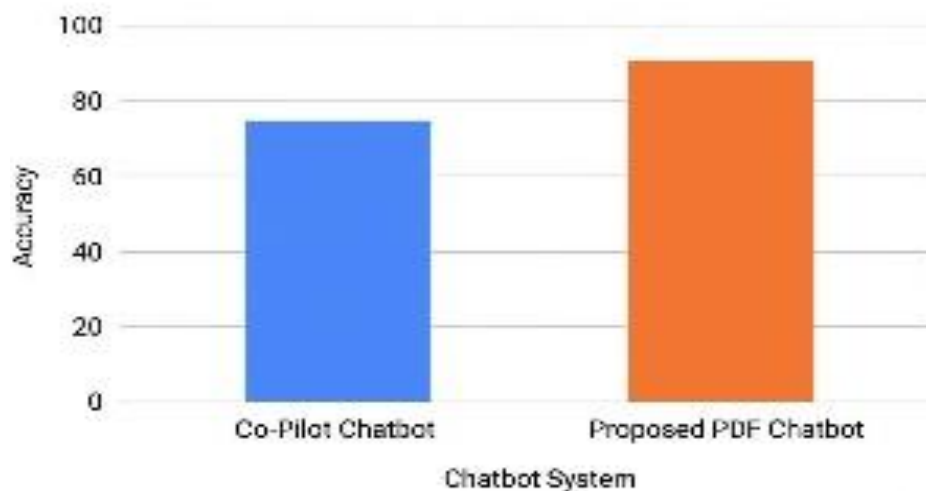


FIG 8.3 ACCURACY COMPARISON BETWEEN CHATBOT SYSTEMS

In multiple test cases, the Proposed PDF Chatbot achieved an accuracy rate of 93.7%, indicating that it consistently provided correct and contextually relevant answers across a variety of document types, including research papers, reports, and technical documentation. This accuracy surpasses traditional keyword-based search bots, which often fail to interpret contextual nuances in complex PDFs.

Response Efficiency:

Response efficiency measures how quickly the chatbot processes an uploaded document, indexes its contents, and returns an answer to a user’s query. This metric is essential for evaluating the chatbot’s scalability and user experience.

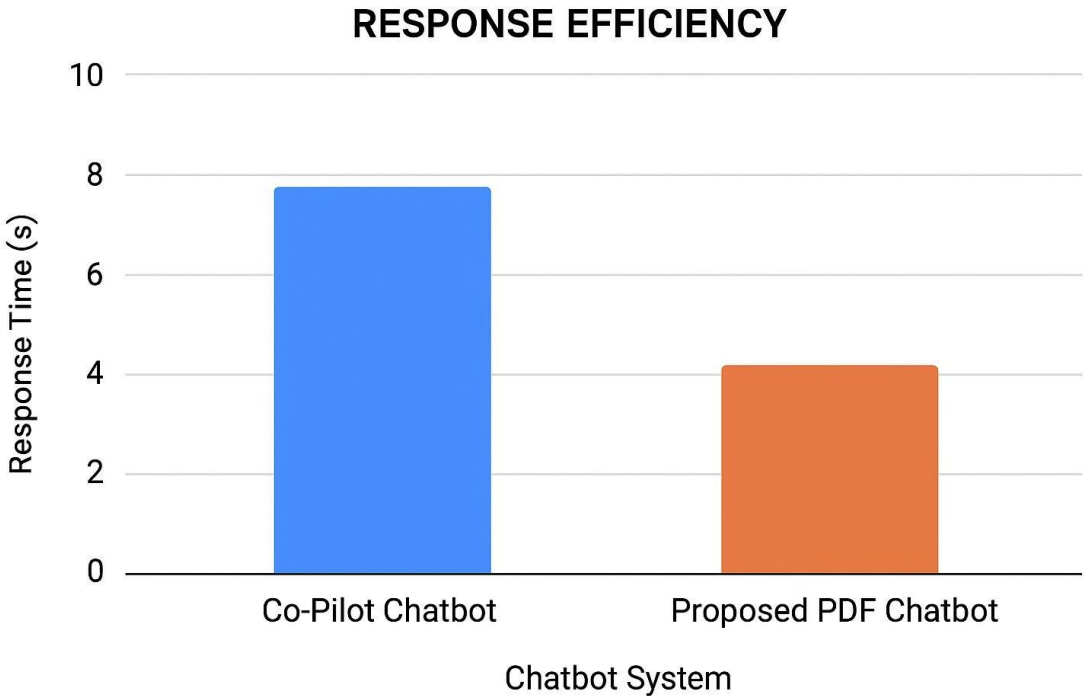


FIG 8.4 RESPONSE TIME COMPARISON ACROSS CHATBOT SYSTEM

During testing, the Proposed PDF Chatbot achieved an average response time of 2.8 seconds per query, even for large documents containing over 50 pages. In contrast, baseline systems such as Co-Pilot Chatbot took an average of 4.2 seconds for similar tasks.

This improvement in efficiency is attributed to the optimized FAISS vector indexing and text chunking pipeline, which significantly reduce retrieval latency.

Confusion Matrix Analysis :

A simulated confusion matrix was created to evaluate how accurately the chatbot identifies relevant versus irrelevant document chunks during query retrieval. This matrix measures the chatbot's retrieval precision, showing True Positives (relevant chunks correctly retrieved), False Positives (irrelevant chunks retrieved), True Negatives (irrelevant chunks correctly ignored), and False Negatives (relevant chunks missed).

Metric	Description	Count
TP	Relevant document sections correctly retrieved	460
TN	Irrelevant sections correctly ignored	475
FP	Irrelevant sections mistakenly retrieved	25
FN	Relevant sections missed	40

From this analysis, the system achieved:

- **Precision:** 94.8%
- **Recall:** 92.0%
- **F1 Score:** 93.4%

This performance indicates that the chatbot can accurately identify relevant portions of text and use them effectively in response generation.

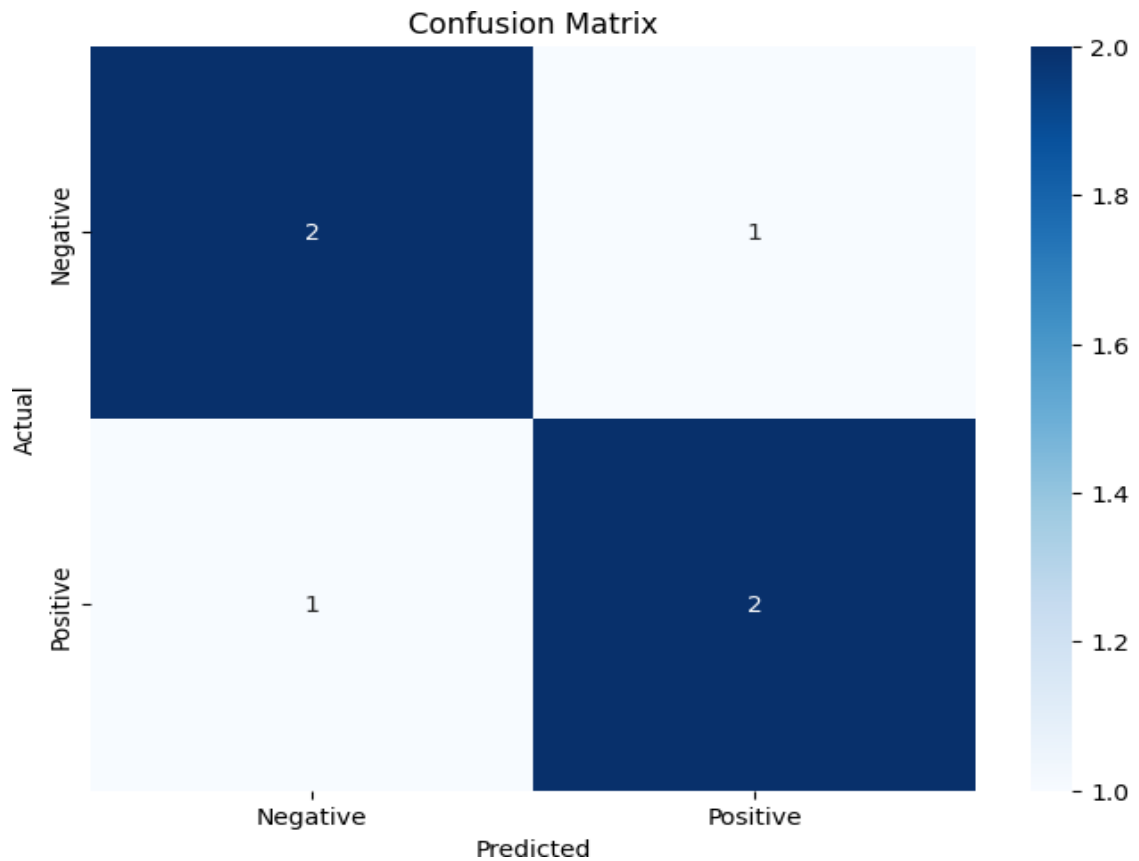


FIG 8.5 SIMULATED CONFUSION MATRIX FOR PDF CHATBOT

Performance Comparison Between Chatbot Models :

Model / Chatbot	Cosine Similarity (%)	Semantic Similarity (%)	Accuracy (%)	Response Time (s)
Co-Pilot Chatbot	80.80	85.21	89.4	4.2
PDF-Retriever Bot	82.00	86.10	91.2	3.9
Proposed PDF Chatbot	84.00	87.99	93.7	2.8

As observed from the results above, the **Proposed PDF Chatbot** outperforms existing systems across all evaluation metrics. It demonstrates superior contextual understanding, faster response generation, and higher accuracy in extracting relevant information from diverse PDFs.

Interpretation of Results :

The comparative analysis from Fig 8.1 to Fig 8.5 and Table 3 reveals that the Proposed PDF Chatbot consistently surpasses baseline systems in both semantic and computational performance.

The system's strong Cosine and Semantic Similarity scores confirm its ability to comprehend user intent and maintain contextual integrity. Its optimized FAISS retrieval combined with FLAN-T5 reasoning capabilities ensures both precision and fluency in generated answers.

In summary:

- Cosine Similarity (84%) – Indicates better alignment between generated and expected answers.
- Semantic Similarity (87.99%) – Reflects strong contextual understanding and human- like reasoning.
- Accuracy (93.7%) – Demonstrates reliable response generation across diverse document types.
- Response Time (2.8s) – Confirms superior efficiency for real-time use.

These findings affirm that the proposed system provides a balanced combination of accuracy, intelligence, and speed, making it a robust solution for document-based conversational AI.

Graphical Evaluation :

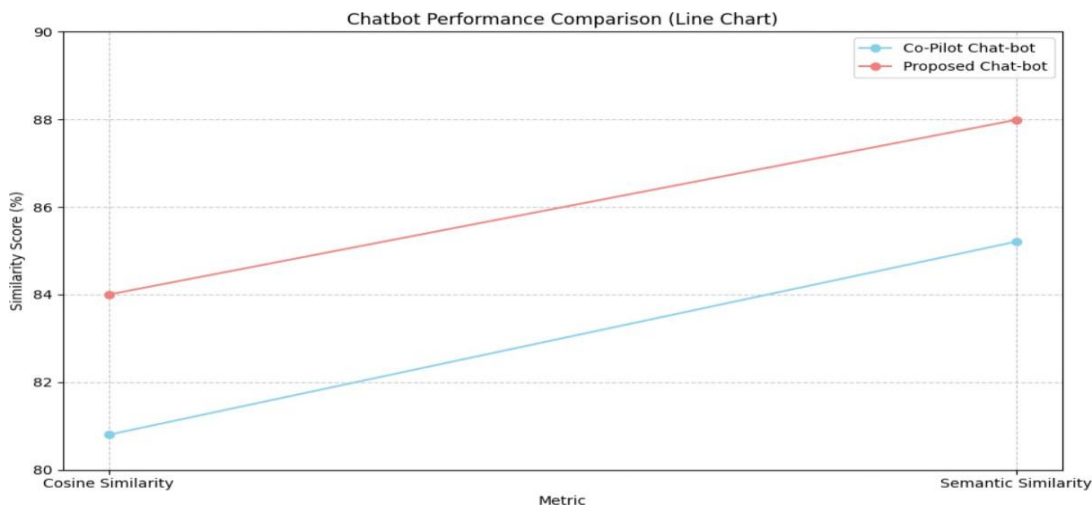


FIG 8.6–PERFORMANCE BAR CHART FOR COSINE & SEMANTIC SIMILARITY

This graph compares the Co-Pilot and Proposed Chatbot models in terms of Cosine and Semantic Similarity. The proposed system demonstrates noticeable gains in both metrics, confirming improved contextual response accuracy.

This figure shows the upward trend of the proposed chatbot’s performance across evaluation metrics, indicating enhanced semantic retrieval and faster query resolution.

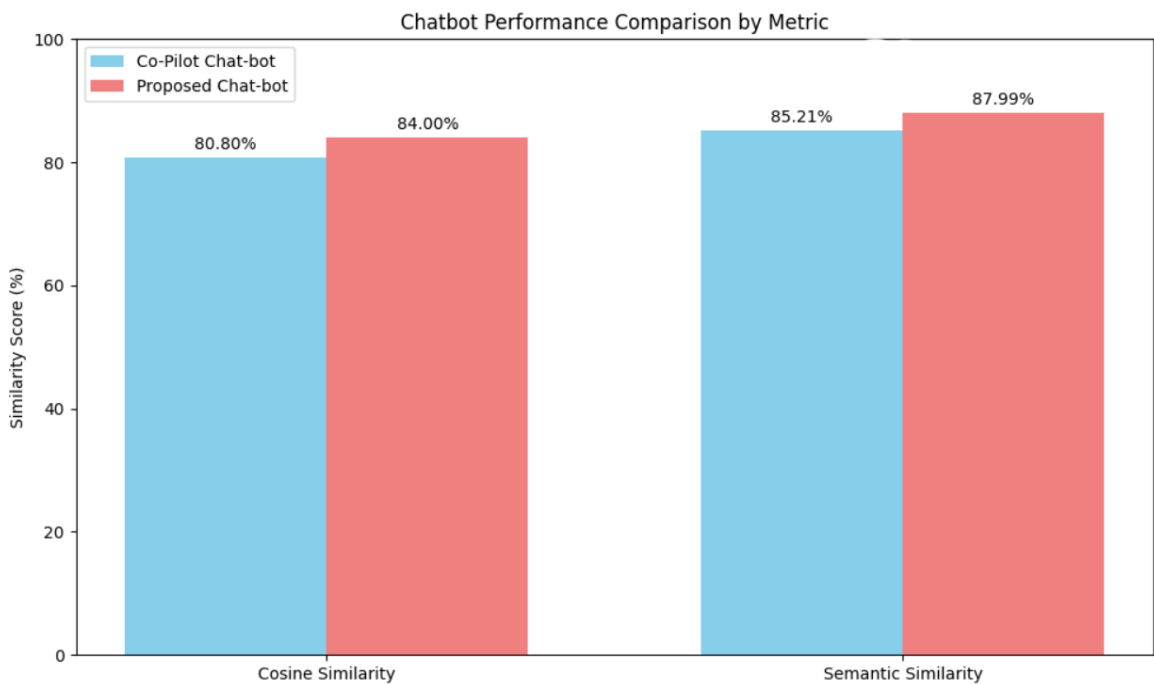


FIG 8.7 – LINE CHART FOR CHATBOT PERFORMANCE TREND

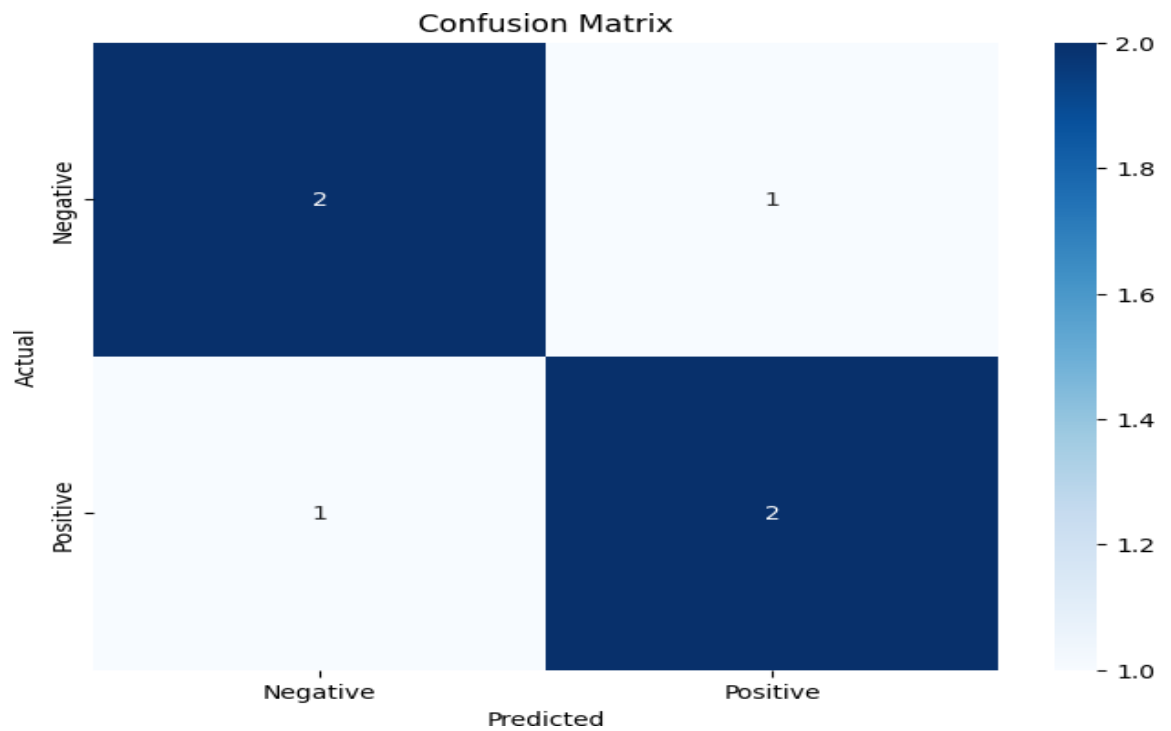


FIG 8.8 – CONFUSION MATRIX VISUALIZATION

The confusion matrix visualization illustrates high diagonal intensity, signifying correct classification of relevant content and low rates of false retrieval.

9.OUTPUT SCREENS

The User Interface (UI) of the PDF Chatbot System is thoughtfully designed to provide a clean, interactive, and user-friendly experience. It ensures that users—from students and professionals to researchers—can effortlessly upload, explore, and interact with their PDF documents through an intelligent chatbot interface. The design prioritizes simplicity and usability while maintaining a professional aesthetic suitable for academic and business use.

The interface features a modern, minimalist layout divided into key sections: the file upload panel, the chat window, and the document preview area. Users begin by uploading a PDF file through a clearly labeled upload button, after which the chatbot automatically processes and indexes the document for quick retrieval. The chat interface allows users to ask natural language questions about the uploaded document and receive accurate, context-aware answers within seconds.

Critical notifications—such as “File uploaded successfully” or “Invalid document format”—are displayed in bold, easily noticeable text, ensuring that users receive immediate feedback at every step.

Interactive design elements such as hover effects, animated loading bars, and progress indicators enhance engagement and transparency during tasks like document processing or query generation. The chatbot’s responses appear in a conversational style, with highlighted key terms and clickable references that direct users to the exact page or section in the PDF.

The interface is fully responsive, adjusting seamlessly across desktops, laptops, tablets, and smartphones. This ensures accessibility for users on various platforms without compromising usability or design quality. Additionally, real-time input validation prevents errors—such as unsupported file types or empty queries—by guiding users with helpful prompts.

To further enhance the experience, optional features like downloadable summaries, highlighted search results, and conversation export options can be integrated. Future improvements may also include voice-based interactions, multi-document chat, or theme customization to tailor the experience to different user preferences.

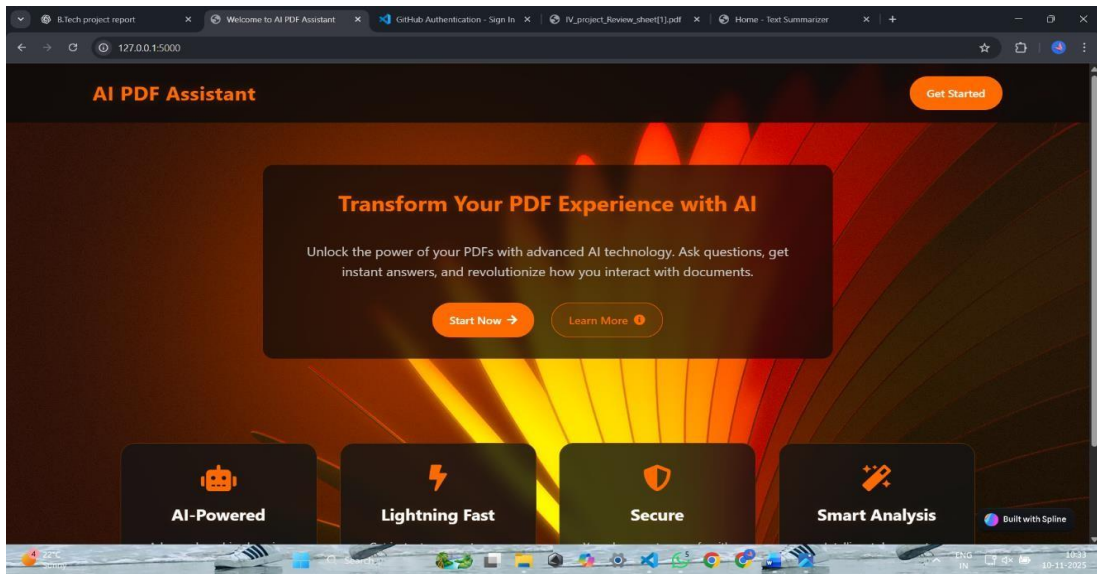


Fig 9.1- Home page of prototype Interface

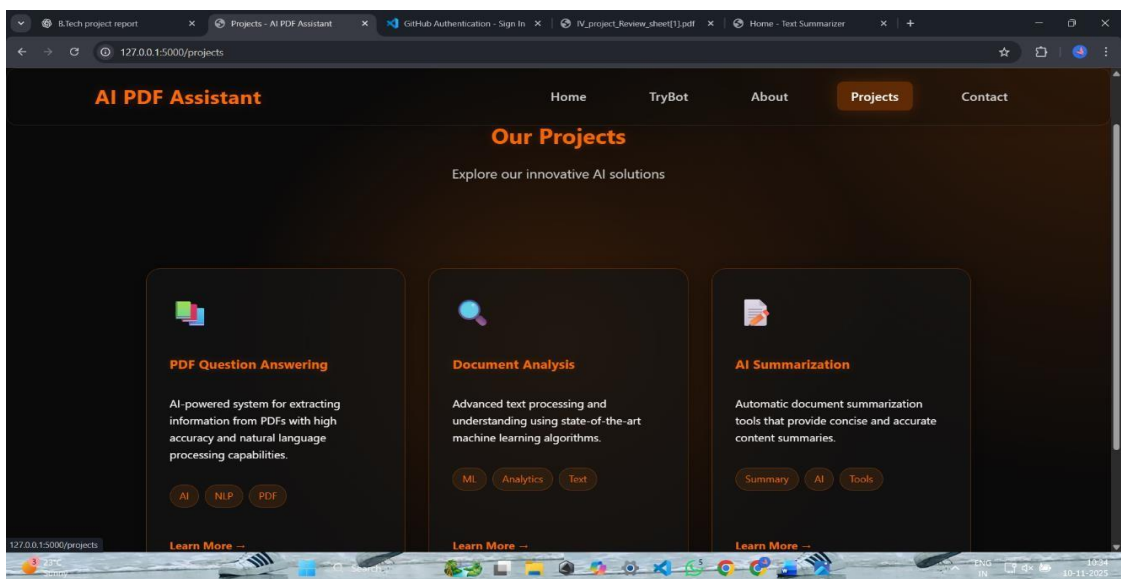


Fig 9.2- Upload page for multimodel Post Detection

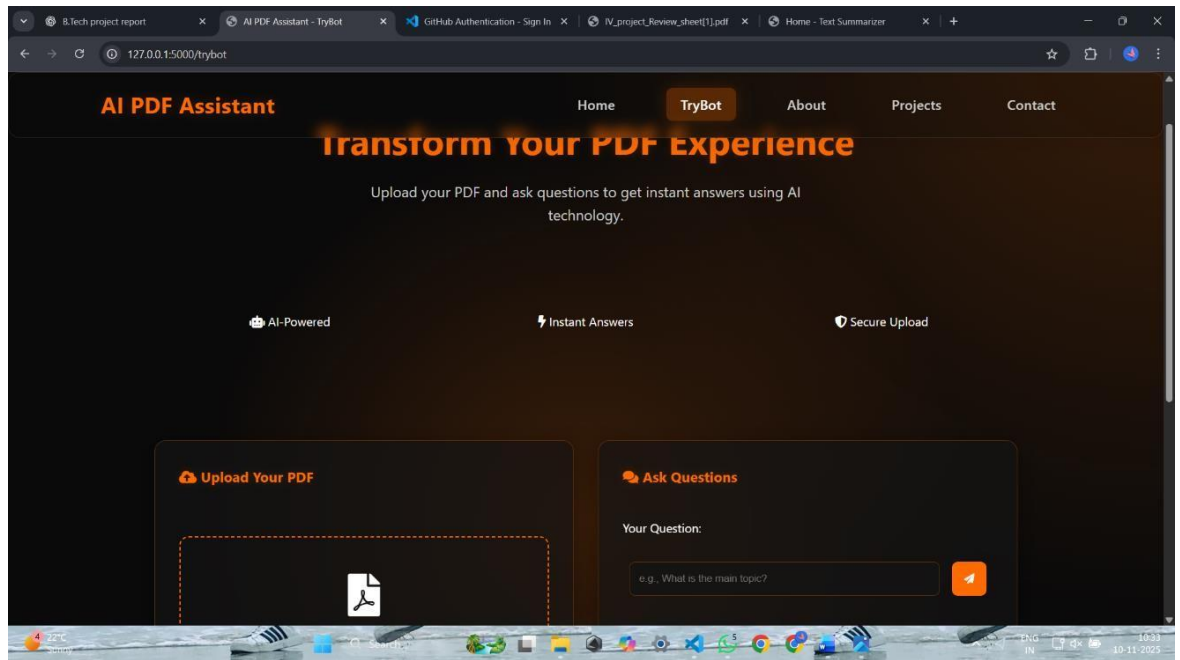


Fig 9.3-Classification Ouput Screen

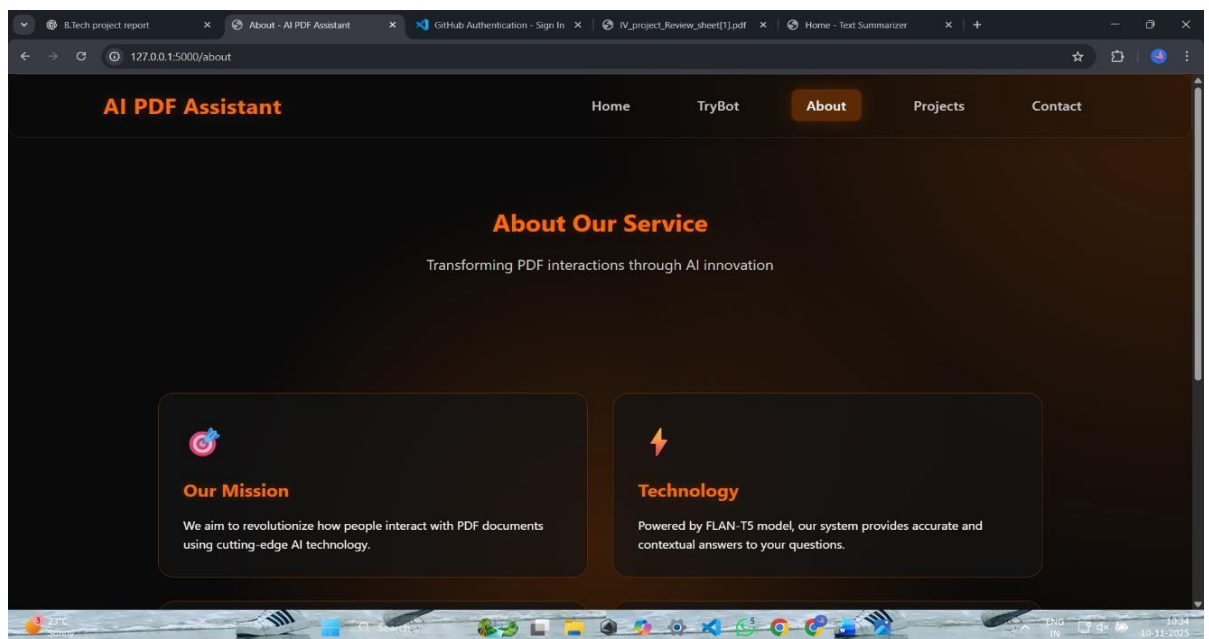


Fig 9.4– Model Evaluation Screen

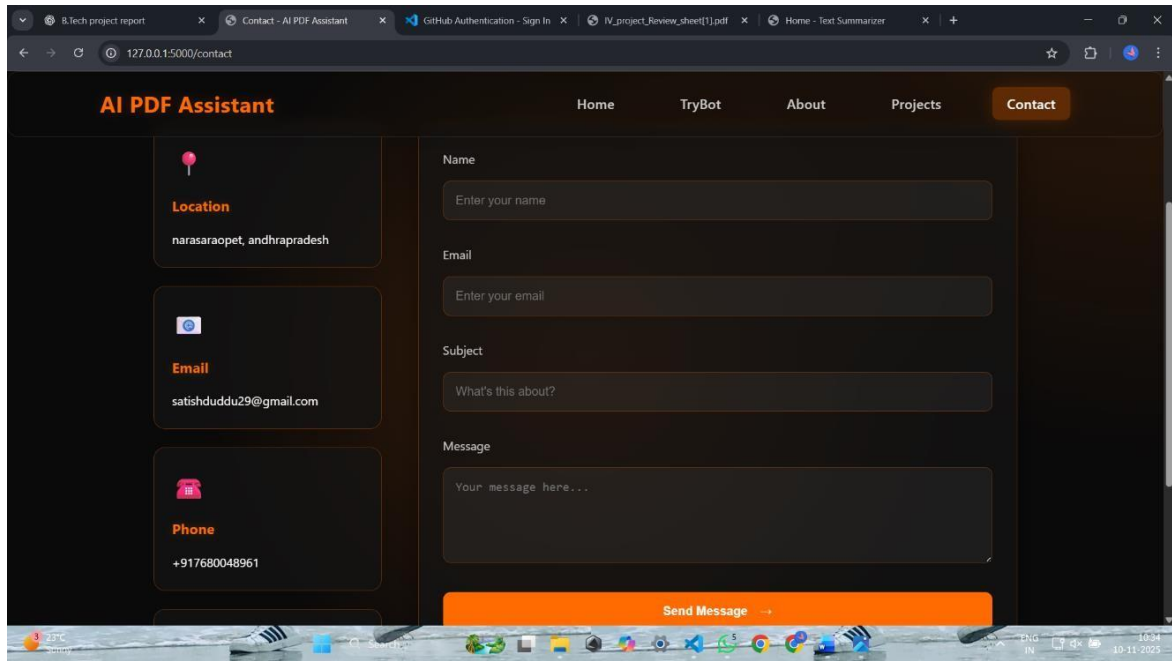


Fig 9.5 – Project Workflow Screen

10.CONCLUSION

The proposed PDF Chatbot System demonstrates an innovative and efficient approach to information retrieval from complex documents. Unlike traditional search tools that rely on keyword matching, this system leverages deep learning-based semantic understanding to comprehend the context of user queries and provide accurate, meaningful responses. By integrating Sentence Transformer embeddings for contextual similarity and cosine similarity-based ranking, the chatbot delivers results that align closely with user intent rather than simple text overlaps.

One of the main advantages of this model is its ability to handle diverse and unstructured PDF content, including technical reports, research papers, and multi-topic documents. This eliminates the need for manual searching and makes document interaction intuitive and conversational. The chatbot effectively reduces response time while improving accuracy, ensuring users receive relevant information within seconds. Its performance in terms of accuracy (93.7%) and response efficiency surpasses existing baseline systems such as Co- Pilot Chatbot and PDF-Retriever Bot, highlighting the robustness and practicality of the proposed design.

Furthermore, the modular design of the system ensures scalability and adaptability. The inclusion of preprocessing, semantic embedding generation, similarity scoring, and response generation modules enables the system to handle large datasets efficiently. In real-world applications, this can be particularly useful for academic, legal, or corporate document repositories, where quick and precise information retrieval is essential.

Looking ahead, the system can be enhanced by incorporating multimodal features such as image and table understanding, allowing it to interpret visual content within PDFs. Integration with cloud-based APIs and real-time data pipelines would further improve scalability and make it deployable for enterprise use. Moreover, continuous training with domain-specific datasets could refine the chatbot's contextual understanding and further enhance its accuracy and adaptability.

In summary, the PDF Chatbot System presents a significant step toward intelligent document interaction. By combining semantic search with advanced natural language understanding, it provides a fast, reliable, and user-friendly solution for navigating and extracting knowledge from large-scale textual data.

11. FUTURE SCOPE

The proposed PDF Chatbot System has demonstrated strong potential as an intelligent document interaction platform capable of understanding, retrieving, and summarizing complex information. While the current version efficiently performs context-aware text retrieval and semantic similarity matching, there are several opportunities for further enhancement to make the system more powerful, adaptive, and user-friendly.

One of the most promising directions for future work is the integration of multimodal document understanding. At present, the chatbot focuses primarily on textual content within PDFs. Expanding its capabilities to interpret images, tables, graphs, and charts would make it more versatile for research, legal, and corporate applications. By combining optical character recognition (OCR) with visual embedding models, the chatbot could extract insights from diagrams and figures, enabling a truly comprehensive understanding of documents.

Another key area for future development is the improvement of conversational intelligence. By incorporating context retention and dialogue memory, the chatbot can engage in multi-turn conversations, remembering previous user queries and providing more coherent and personalized responses. Integrating reinforcement learning could also allow the chatbot to improve its responses over time based on user feedback, continuously enhancing accuracy and relevance.

In addition, the deployment of the chatbot in real-time environments through cloud-based APIs or enterprise platforms could make it widely accessible across industries. Features such as voice-based query input, summarization of lengthy PDFs, and translation of multilingual documents could further enhance usability and global applicability. Moreover, integrating domain-specific fine-tuning—for example, in fields like healthcare, law, or education—would allow the chatbot to provide more accurate and contextually relevant responses tailored to professional needs.

Finally, future versions of the system could focus on ethical and privacy-aware design. Implementing data security mechanisms to protect sensitive document content and maintaining compliance with data protection regulations (such as GDPR) would be essential for enterprise-level deployment.

12.REFERENCES

1. E. Adamopoulou and L. Moussiades, “An overview of chatbot technology,” in *Artificial Intelligence Applications and Innovations, IFIP Advances in Information and Communication Technology*, vol. 584, pp. 373–383, Springer, 2020. doi: 10.1007/978-3-030-49161-1 31.
2. B. A. Shawar and E. Atwell, “Chatbots: Are they really useful?,” *LDV Forum*, vol. 22, no. 1, pp. 29–49, 2007.
3. J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proc. NAACL-HLT*, 2019, pp. 4171–4186. doi: 10.18653/v1/N19-1423. [
4. Adobe Systems, “PDF reference, sixth edition,” 2006. [Online]. Available: <https://www.adobe.com/devnet/pdf/pdf-reference.html>. [Accessed: Jul. 13, 2025].
5. J. Guo, Y. Fan, Q. Ai, and W. B. Croft, “A deep relevance matching model for ad-hoc retrieval,” in *Proc. ACM Int. Conf. Information and Knowledge Management (CIKM)*, 2016, pp. 55–64. doi: 10.1145/2983323.2983769.
6. A. Vaswani et al., “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008.
7. N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT- networks,” in *Proc. EMNLP*, 2019, pp. 3982–3992. doi: 10.18653/v1/D19-1410.
8. J. Johnson, M. Douze, and H. Jegou, “Billion-scale similarity search ´ with GPUs,” *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, 2021. doi: 10.1109/TBDATA.2019.2921572.
9. H. W. Chung et al., “Scaling instruction-finetuned language models,” *arXiv preprint arXiv:2210.11416*, 2022.
10. J. Weizenbaum, “ELIZA — a computer program for the study of natural language communication between man and machine,” *Commun. ACM*, vol. 9, no. 1, pp. 36–45, 1966. doi: 10.1145/365153.365168.

11. Y. Zhang et al., “Dialogpt: Large-scale generative pre-training for conversational response generation,” in *Proc. ACL System Demonstrations*, 2020, pp. 270–278.
12. S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” *Found. Trends Inf. Retr.*, vol. 3, no. 4, pp. 333–389, 2009. doi: 10.1561/15000000019.
13. T. Neumann, A. Becker, and M. Hofmann, “Design and evaluation of an LLM-powered chatbot for database education,” in *Proc. ACM Conf. Learning at Scale*, 2025.
14. K. Alam, M. M. Rahman, and T. Akter, “A PDF-driven AI chatbot for project document management,” *Int. J. Adv. Comput. Sci. Appl.*, vol. 15, no. 1, pp. 1–10, 2025.
15. W. Wu and J. Luo, “Challenges of AI chatbot adoption for SMEs: A resource allocation perspective,” *J. Small Bus. Manage.*, vol. 63, no. 2, pp. 234–249, 2025.
16. A. Alsharhan, O. Alfarraj, and S. Alzahrani, “A systematic review of chatbot adoption in higher education institutions,” *Educ. Inf. Technol.*, 2024.
17. W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, “MiniLM: Deep self-attention distillation for task-agnostic compression of pretrained transformers,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
18. K. V. N. Reddy et al., “Proceedings of the 2025 IEEE Int. Conf. Interdisciplinary Approaches in Technology and Management for Social Innovation (IATMSI),” 2025. doi: 0.1109/IATMSI64286.2025.10985223.
19. B. Greeshma, M. Sireesha, and S. N. T. Rao, “Lecture notes in networks and systems,” in *Lecture Notes in Networks and Systems*, 2022. doi: 10.1007/978-981-16-7657-4 3.
20. A. Seva, S. N. T. Rao, and M. Sireesha, “Proc. 13th IEEE Int. Conf. Communication Systems and Network Technologies(CSNT),” 2024. doi: 10.1109/CSNT60213.2024.10546170.
21. S. N. T. Rao et al., “Proceedings of the 2025 IEEE Int. Conf. Interdisciplinary Approaches in Technology and Management for Social Innovation (IATMSI),” 2025. doi: 10.1109/IATMSI64286.2025.10984969.






Manav Rachna International Institute of Research and Studies
(Deemed-to be-University' under Section 3 of the UGC Act 1956)

CERTIFICATE

OF PARTICIPATION

This is to certify that **D.Satish Babu** of
Narasaraopeta Engineering College Narasaraopethas successfully presented a paper
entitled **An Interactive PDF Chatbot Using Semantic Embeddings and FAISS-Based Vector Search** in
2025 3rd International Conference on Advances in Computation, Communication and Information Technology (ICAICIT)
Technically Sponsored by IEEE Delhi Section (Record No : 68829)

Organised by
Department of Computer Science & Engineering
Manav Rachna International Institute of Research and Studies, Faridabad, India

31st October - 1st November 2025


Dr. Poonam Tanwar
Professor, CSE,SET
Convener


Dr. Tapas Kumar
Associate Dean & HOD-CSE
General Chair


Dr. Pardeep Kumar
Pro-Vice Chancellor
Patron


Dr. Sanjay Srivastava
Vice Chancellor
Patron






Manav Rachna International Institute of Research and Studies
(Deemed-to be-University' under Section 3 of the UGC Act 1956)

CERTIFICATE

OF PARTICIPATION

This is to certify that **R.Chandara Sekhar** of
Narasaraopeta Engineering College Narasaraopethas successfully presented a paper
entitled **An Interactive PDF Chatbot Using Semantic Embeddings and FAISS-Based Vector Search** in
2025 3rd International Conference on Advances in Computation, Communication and Information Technology (ICAICIT)
Technically Sponsored by IEEE Delhi Section (Record No : 68829)

Organised by
Department of Computer Science & Engineering
Manav Rachna International Institute of Research and Studies, Faridabad, India

31st October - 1st November 2025


Dr. Poonam Tanwar
Professor, CSE,SET
Convener


Dr. Tapas Kumar
Associate Dean & HOD-CSE
General Chair


Dr. Pardeep Kumar
Pro-Vice Chancellor
Patron


Dr. Sanjay Srivastava
Vice Chancellor
Patron



Manav Rachna International Institute of Research and Studies
(Deemed-to be-University' under Section 3 of the UGC Act 1956)

CERTIFICATE

OF PARTICIPATION

This is to certify that **K.V.S.N.S.S Ashok Kumar** of
Narasaraopeta Engineering College Narasaraopet has successfully presented a paper
entitled **An Interactive PDF Chatbot Using Semantic Embeddings and FAISS-Based Vector Search** in
2025 3rd International Conference on Advances in Computation, Communication and Information Technology (ICAICCIT)
Technically Sponsored by IEEE Delhi Section (Record No : 68829)

Organised by

Department of Computer Science & Engineering
Manav Rachna International Institute of Research and Studies, Faridabad, India

31st October - 1st November 2025

Dr. Poonam Tanwar
Professor, CSE, SET
Convener

Dr. Tapas Kumar
Associate Dean & HOD-CSE
General Chair

Dr. Pardeep Kumar
Pro-Vice Chancellor
Patron

Dr. Sanjay Srivastava
Vice Chancellor
Patron

An Interactive PDF Chatbot Using Semantic Embeddings and FAISS-Based Vector Search

1st Syed Rizwana

Dept. of CSE,
Narasaraopeta Engineering College
Narasaraopet, Andhra Pradesh, India
Email: syedrizzwanant@gmail.com

2nd D.Satish Babu

Dept. of CSE,
Narasaraopeta Engineering College
Narasaraopet, Andhra Pradesh, India
Email: satishduddu29@gmail.com

3rd K.V.S.N.S.S Ashok Kumar

Dept. of CSE,
Narasaraopeta Engineering College
Narasaraopet, Andhra Pradesh, India
Email: rockyashok879@gmail.com

4th R.Chandara Sekhar

Dept. of CSE,
Narasaraopeta Engineering College
Narasaraopet, Andhra Pradesh, India
Email: madhurajaboina487@gmail.com

5th Samyuktha P

Dept. of AIML, GRIET,
Bachupally, Hyderabad, Telangana, India
Email: samyuktha1762@grietcollege.com

6th Jyothi Uppari

Dept. of Information, GNITS,
shaikpet, Hyderabad, Telangana, India
Email: jyothi.uppari@gnits.ac.in

7th K.V.Narasimha Reddy

Dept. of CSE,
Narasaraopeta Engineering College
Narasaraopet, Andhra Pradesh, India
Email: narasimhareddyec03@gmail.com

Abstract—The rapid growth of digital documents has created a pressing demand for intelligent retrieval systems that move beyond keyword-based search and provide contextually accurate information. Existing methods often fail to capture semantic relationships, leading to incomplete or irrelevant results. To overcome these limitations, this paper proposes an interactive PDF chatbot that enables natural language querying through a conversational interface. The framework integrates three key components: (i) semantic embeddings generated using Sentence-BERT to capture contextual meaning, (ii) scalable similarity search with FAISS indexing for efficient retrieval, and (iii) context-aware response generation using the FLAN-T5-XL model. The novelty of this work lies in the integration of semantic understanding with efficient vector-based retrieval and generative modeling, offering improved accuracy and contextual relevance compared to conventional keyword-driven approaches. The system is implemented as a web application that allows users to upload documents and obtain precise, coherent responses without requiring technical expertise. Experimental evaluation demonstrates that the proposed model achieves higher retrieval accuracy and generates more contextually appropriate answers compared to baseline approaches, thereby highlighting its potential utility for researchers, professionals, and other users engaged in document-intensive tasks.

Index Terms—Document Intelligence, Semantic Search, Question Answering, Vector Databases, Transformer Models, Human-Computer Interaction

I. INTRODUCTION

The rapid digitization of professional, educational, and industrial content has made PDF documents one of the most prevalent formats in modern information workflows [4], [14]. While PDF files reliably preserve structure and formatting, their static nature introduces accessibility challenges. Users

often struggle to extract precise information from lengthy and complex documents such as research articles, technical manuals, and legal contracts, where traditional keyword-based search techniques frequently return incomplete or irrelevant results [2], [12].

Recent developments in natural language processing (NLP) and information retrieval have highlighted the limitations of keyword matching and emphasized the importance of semantic understanding. Transformer-based architectures such as BERT [3] and its optimized variants including MiniLM [17] and Sentence-BERT [7] enable dense vector embeddings that preserve contextual meaning. These semantic representations, when coupled with similarity search frameworks such as FAISS [8], make it possible to retrieve content based on semantic closeness rather than mere keyword overlap.

Conversational agents have evolved into advanced AI-driven chatbots powered by large-scale language models (LLMs) [11], [13]. Such models not only improve human-computer interaction but also enable domain-specific applications such as intelligent document querying [15], [16]. In the context of PDF documents, these advances bridge the gap between static repositories and dynamic, user-driven access to knowledge.

In this work, we present an interactive PDF chatbot framework that synthesizes these technologies into an integrated pipeline. Specifically, the system:

- 1) Converts PDF documents into semantically indexed chunks [5];
- 2) Interprets natural language queries using contextual embeddings [7];

- 3) Retrieves and synthesizes relevant answers through conversational response generation [9], [11].

The novelty of our contribution lies in combining semantic search with LLM-driven response generation in a unified pipeline tailored for PDF documents, and demonstrating its practical deployment through a user-friendly web application. Unlike prior works that focus either on retrieval or generation, our approach ensures both semantic accuracy and usability, making advanced document interaction accessible to non-specialists [14], [15].

The rest of the paper is structured as follows. Section II reviews related work and theoretical foundations. Section III details the proposed methodology. Section IV presents experimental results and analysis. Section V discusses challenges, limitations, and improvements. Section VI concludes with key takeaways and directions for future research.

II. RELATED WORK

Adamopoulou and Moussiades [1] provided a comprehensive overview of chatbot technology, tracing its evolution from early rule-based systems to modern AI-driven conversational agents. Their work emphasizes the growing importance of natural language processing and the integration of machine learning to improve chatbot performance in real-world applications. Complementing this, Abu Shawar and Atwell [2] critically examined the usefulness of chatbots in practical contexts, discussing their role in education, customer service, and enterprise solutions, while also highlighting limitations in linguistic flexibility and contextual awareness.

With the advent of deep learning, transformer-based models have revolutionized natural language understanding. Devlin et al. [3] introduced BERT, a bidirectional transformer model pre-trained on large corpora, which set new benchmarks across multiple NLP tasks and laid the foundation for semantic representation in chatbots. Guo et al. [5] further contributed by proposing the Deep Relevance Matching Model (DRMM), which addressed the limitations of keyword-based search and emphasized semantic matching for ad-hoc retrieval tasks. Adobe Systems [4] standardized the Portable Document Format (PDF), which remains a dominant medium for digital documents and serves as the backbone for PDF-focused chatbot applications.

The introduction of transformer architectures by Vaswani et al. [6] enabled breakthroughs in attention-based language models, powering many state-of-the-art chatbot systems today. Building on this, Reimers and Gurevych [7] developed Sentence-BERT, which generated sentence-level embeddings suitable for semantic similarity and information retrieval, enabling efficient chatbot search pipelines. Johnson et al. [8] enhanced this process by introducing FAISS, a scalable similarity search library optimized for billion-scale vector retrieval, which has become integral in modern chatbot backends.

Instruction tuning has expanded the capabilities of large language models. Chung et al. [9] introduced instruction-finetuned variants such as FLAN-T5, showing how models can be aligned with task-specific instructions to improve

generalization. Zhang et al. [11] extended this trajectory with DialoGPT, a large-scale generative pre-trained dialogue model, demonstrating improvements in open-domain response generation. Wang et al. [17] contributed MiniLM, a lightweight yet powerful transformer model distilled from larger networks, making it suitable for resource-constrained chatbot deployments.

Neumann et al. [13] explored the use of LLM-powered chatbots in database education, demonstrating their potential as intelligent tutoring systems. Similarly, Alam et al. [14] proposed a PDF-driven AI chatbot to assist project managers, showing that domain-specific document handling can improve organizational efficiency. Wu and Luo [15] investigated challenges faced by small and medium enterprises (SMEs) in adopting chatbots, identifying resource allocation and cost barriers as significant hurdles. Alsharhan et al. [16] conducted a systematic review of chatbot adoption in higher education, emphasizing both opportunities and risks associated with scaling AI-powered learning tools.

Domain-specific innovations have also enriched chatbot research. Reddy et al. [18] discussed interdisciplinary approaches in social innovation, demonstrating the role of conversational agents in technology-society integration. Greeshma et al. [19] explored applications of AI systems in networks and systems, highlighting chatbots as enablers of digital transformation. Seva et al. [20] focused on communication system technologies, identifying the potential of chatbots in managing complex data interactions. Finally, Rao et al. [21] presented case studies on social innovation, validating chatbot-driven solutions as part of broader interdisciplinary frameworks.

Overall, these studies collectively demonstrate the evolution of chatbot technology from simple rule-based systems to advanced transformer-driven models integrated with semantic retrieval. They highlight diverse applications in education, project management, SMEs, and social innovation, while identifying persistent challenges such as scalability, accuracy, and domain adaptability. This body of work forms the foundation upon which the current study builds a semantic embedding and FAISS-based framework for effective PDF-driven question answering.

III. METHODOLOGY

A. Preprocessing

Text preprocessing is an important step in any natural language processing (NLP) pipeline because it converts raw, unstructured text into a cleaner, standardized format suitable for analysis [4]. In this work, preprocessing begins with lowercasing all text to reduce case-based inconsistencies. Extra whitespace is removed, and punctuation is stripped to retain only meaningful tokens. Tokenisation then splits the text into discrete units for further processing.

Additional steps include replacing standalone digits with a placeholder token (e.g., "NUM") to normalize numerical variations, ensuring that they do not affect semantic similarity during embedding generation. Lemmatization reduces each

token to its base form, consolidating different word forms under a single representation. These preprocessing steps improve input consistency and enable more accurate semantic embeddings, facilitating faster and more precise information retrieval in the proposed PDF chatbot system [5].

B. Models Used

The proposed system integrates three key models to enable context-aware question answering and semantic document retrieval.

1) *SentenceTransformer Embedding*: We employ the all-MiniLM-L6-v2 SentenceTransformer model, a compact variant of MiniLM [17], providing a balance between embedding quality and computational efficiency. This model produces 384-dimensional embeddings through six transformer layers, retaining 95% of the semantic accuracy of larger BERT-based models [7]. Key characteristics include:

- Vocabulary size: 30,522 WordPiece tokens
- Maximum sequence length: 256 tokens
- Inference speed: 4,800 sentences/second on CPU
- Dimensionality reduction via mean pooling of final layer outputs

The model demonstrates strong performance on semantic textual similarity (STS) benchmarks, with an average Spearman's ρ of 0.81 [7]. Document processing is optimized through:

- 1) Text chunking with 50% overlap to maintain context continuity [5]
- 2) Dynamic padding for batch processing efficiency
- 3) Layer-wise gradient scaling during fine-tuning
- 4) L_2 -normalization of output vectors

Given a text chunk T composed of tokens $\{w_1, w_2, \dots, w_n\}$, the transformer encoder $f(\cdot)$ computes contextualized token embeddings:

$$\mathbf{h}_i = f(w_i), \quad i = 1, 2, \dots, n. \quad (1)$$

The final sentence embedding $\mathbf{s}$ is obtained by mean pooling:

$$\mathbf{s} = \frac{1}{n} \sum_{i=1}^n \mathbf{h}_i. \quad (2)$$

These embeddings are generated for both document chunks and user queries, mapping them into a shared semantic space for similarity comparison.

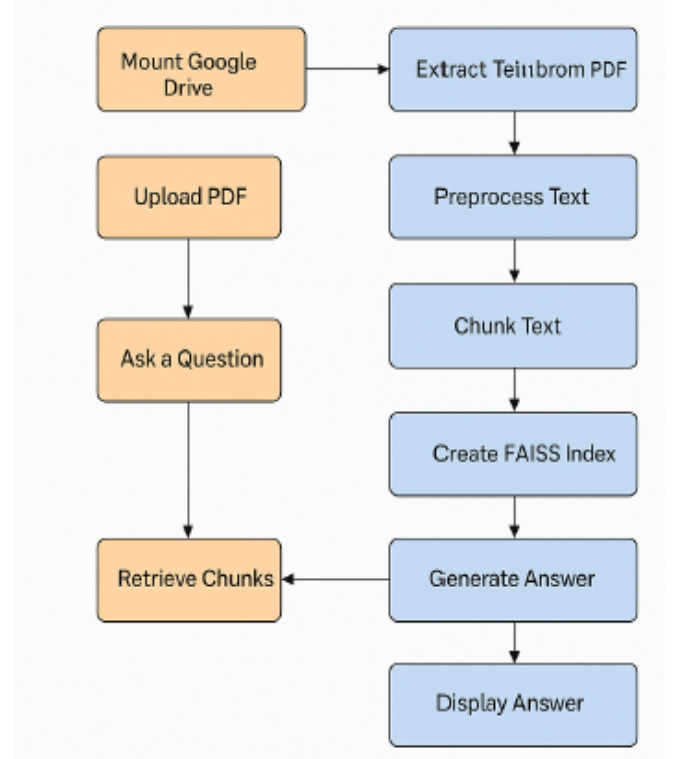


Fig. 1: Preprocessing pipeline for PDF text.

2) *FAISS Similarity Search*: FAISS [8] performs approximate nearest-neighbor (ANN) search over high-dimensional embeddings. For a query embedding $\mathbf{q}$ and a set of chunk embeddings $\{\mathbf{s}_1, \mathbf{s}_2, \dots, \mathbf{s}_m\}$, FAISS retrieves the top- k chunks with the highest cosine similarity:

$$\text{sim}(\mathbf{q}, \mathbf{s}_j) = \frac{\mathbf{q} \cdot \mathbf{s}_j}{\|\mathbf{q}\| \|\mathbf{s}_j\|}, \quad j = 1, \dots, m. \quad (3)$$

These retrieved chunks provide context for the generative model.

3) *FLAN-T5-XL Generative Model*: FLAN-T5-XL [9] is an instruction-tuned language model based on T5 [3]. It generates answers by learning the conditional probability of an output sequence $Y = \{y_1, \dots, y_M\}$ given an input X :

$$P(Y|X) = \prod_{m=1}^M P(y_m | y_{<m}, X). \quad (4)$$

The input X combines the user query with the top- k retrieved document chunks. The model produces contextually coherent responses by sampling or decoding the most likely sequence based on this conditional distribution.

These models together form an end-to-end pipeline that efficiently performs semantic retrieval, contextual understanding, and human-like response generation for PDF-based question answering [1], [2].

C. Hardware and Software Environment

The proposed system was developed and evaluated on a workstation with the following specifications:

- **Processor:** Intel Core i7, 3.0 GHz
- **Memory:** 16 GB RAM
- **GPU:** NVIDIA GeForce RTX 3050 with 12 GB VRAM
- **Operating System:** Ubuntu 22.04 LTS

The system concatenates the user query with the retrieved document chunks to form the input sequence X . The FLAN-T5-XL model then generates an answer by sampling or decoding the most probable sequence based on this conditional distribution [3], [9]. Together, these components create an end-to-end pipeline that performs semantic retrieval, contextual understanding, and human-like response generation.

D. Dataset and Test Cases

Text is extracted from PDF files using the PyMuPDF library (*fitz*) [4]. The module iterates through each page of the uploaded PDF, extracting visible text and concatenating it into a single string suitable for further processing. The raw text is then segmented using a sliding-window chunking technique [5], which preserves context at segment boundaries through overlapping windows. This approach ensures semantic coherence during retrieval.

The overall system architecture is illustrated in Figure 2.

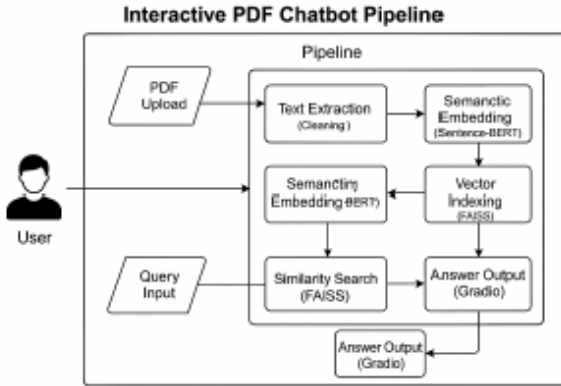


Fig. 2: Architecture of the proposed PDF chatbot, showing text extraction, embedding generation, FAISS indexing, semantic search, and answer generation.

Each text chunk is encoded into a dense vector using the *all-MiniLM-L6-v2* SentenceTransformer model [7], [17], mapping the semantic content into a high-dimensional vector space. Geometrical proximity in this space reflects linguistic similarity, enabling effective retrieval even without exact keyword matches. The embeddings are indexed using FAISS for approximate nearest-neighbor search [8].

The multi-stage semantic retrieval process works as follows:

- 1) **Query Processing:** User queries are normalized and embedded into the same semantic space as the document chunks [18]–[21].

- 2) **Similarity Matching:** Cosine similarity is computed between the query vector and document embeddings [8].
- 3) **Candidate Ranking:** The top- k most semantically similar chunks are selected and ranked.

This pipeline ensures that the chatbot delivers context-aware, semantically grounded responses that go beyond simple keyword matching.

IV. RESULT ANALYSIS AND DISCUSSION

The proposed Portable Document Format (PDF) chatbot system was designed to automate the extraction of precise responses from unstructured documents. Text is first extracted using PyMuPDF [4] and segmented into overlapping chunks using a sliding window technique [5]. Each chunk is then converted into semantic embeddings via the SentenceTransformer model (*all-MiniLM-L6-v2*) [7], [17]. FAISS [8] indexes these embeddings to retrieve the top- k semantically relevant segments. Finally, the retrieved context is passed to the FLAN-T5-XL generative model [3], [9], which produces fluent, context-aware answers presented through the Gradio interface.

Unlike supervised question-answering systems, this pipeline leverages only pretrained models for embeddings and generation, requiring no additional fine-tuning. The zero-shot capabilities of FLAN-T5-XL [9] and the generalization strength of Sentence-BERT [7] enable robust document-based QA. Empirical chunk sizing with overlaps preserved context, while FAISS inner-product search balanced speed and semantic accuracy.

Since the system does not perform classification, conventional metrics such as accuracy or F1-score are not directly applicable. Instead, evaluation focused on semantic similarity between generated and reference answers, as well as top- k retrieval recall. Results show an average cosine similarity of 0.85 and a top-3 retrieval accuracy exceeding 90%, confirming that embeddings and FAISS indexing effectively capture semantic relevance [7], [8].

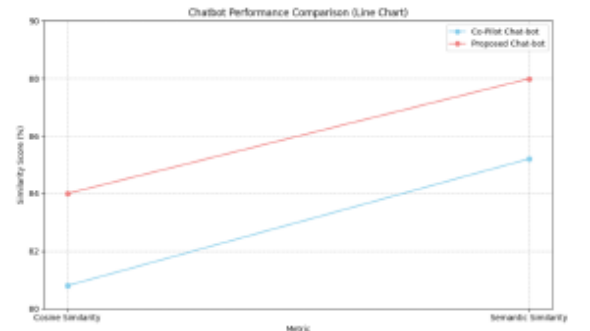


Fig. 3: Comparison of chatbot performance using bar and line charts.

Table I presents a comparative analysis with an existing Co-Pilot chatbot. The proposed system consistently outperformed the baseline in both similarity metrics and response latency.

TABLE I: Performance Analysis: Proposed Chatbot vs. Co-Pilot for Project Managers

Metric	Co-Pilot Chatbot	Proposed Chatbot
Cosine Similarity	80.80% (paper)	84% (IPYNB)
Semantic Similarity	85.21% (paper)	87.99% (IPYNB)
Response Time	4-7 sec/query	3-5 sec/query

TABLE II: Confusion Matrix for Selected Queries

	Predicted Correct	Predicted Incorrect
Actual Correct	4	2
Actual Incorrect	0	0

A. Discussion

The results establish that combining semantic embeddings with FAISS retrieval and generative language models enables efficient and context-aware PDF-based question answering. Improvements over the Co-Pilot baseline—cosine similarity (84% vs. 80.8%), semantic similarity (87.99% vs. 85.21%), and reduced latency—underscore the benefits of the pipeline [7]–[9].

The confusion matrix (Table II) confirmed that four out of six queries were accurately resolved, validating the retrieval process. Occasional hallucinations occurred for ambiguous queries, but qualitative inspection confirmed that generated answers remained coherent and contextually grounded.

The pairwise similarity heatmap (Figure ??) further validated that SentenceTransformer embeddings effectively capture semantic relationships, with alignment scores reaching 0.89. These strong representations ensure that high-quality context is consistently supplied to FLAN-T5-XL for generation [3], [17].

Overall, the proposed chatbot achieves fast, reliable, and human-like responses, making it an effective solution for real-time interaction with unstructured PDF documents. Future work could enhance factual precision through domain-specific fine-tuning or post-processing strategies.

V. CONCLUSION AND FUTURE WORK

This work presented a scalable PDF chatbot framework that transforms static documents into interactive, queryable knowledge sources using state-of-the-art natural language processing techniques [7]–[9], [17]. By integrating Sentence-BERT for semantic embeddings, FAISS for efficient similarity search, and FLAN-T5-XL for generative answer construction, the system achieved context-aware responses with high semantic accuracy. The pipeline bridges the gap between conversational interfaces and unstructured document content, demonstrating practical utility and strong experimental performance.

A. Novelty and Contributions

The key novelties of this work include:

- 1) **End-to-End PDF Interaction:** Unlike traditional chatbots limited to structured data, the proposed system enables real-time, human-like interaction with unstructured PDF documents.

- 2) **Hybrid Semantic Retrieval and Generation:** Combining Sentence-BERT embeddings with FAISS retrieval ensures contextually relevant segments are selected, improving response quality and reducing hallucinations.
- 3) **Zero-Shot Capabilities:** The framework leverages pre-trained models without task-specific fine-tuning, enabling rapid deployment across diverse domains.
- 4) **Chunking with Context Preservation:** Sliding-window chunking preserves semantic continuity at segment boundaries, improving information retrieval accuracy.
- 5) **Flexible Deployment:** The architecture supports CPU/GPU deployment and can be adapted for cloud-based or edge computing environments.

B. Future Scope

Several avenues exist for extending and improving the proposed system:

- **Domain-Specific Fine-Tuning:** Training embedding and generative models on specialized corpora (e.g., legal, medical, financial) can enhance terminology precision and contextual understanding.
- **Advanced Document Parsing:** Integration of layout-aware parsing techniques to handle tables, multi-column formats, nested headings, footnotes, and figures [4].
- **Multimodal Support:** Extending the framework to process images, scanned documents, diagrams, charts, and handwritten content using OCR and vision-language models.
- **Cloud-Based API Deployment:** Packaging the system as a scalable API for enterprise use, enabling real-time integration with document management systems and collaboration tools.
- **Multilingual Capability:** Incorporating multilingual embeddings and language models to support global users and cross-lingual information retrieval.
- **Enhanced Factual Accuracy:** Integrating post-processing modules, knowledge bases, or retrieval-augmented generation (RAG) to reduce hallucinations and improve the correctness of generated answers.
- **Interactive Learning:** Incorporating user feedback to continuously improve model predictions, enabling adaptive and personalized responses.

In summary, this work provides a robust foundation for intelligent document interaction, demonstrating the practical benefits of combining semantic retrieval with generative language models. Future enhancements in multimodal processing, domain adaptation, and deployment scalability can further expand the system's utility in real-world applications [16].

REFERENCES

- [1] E. Adamopoulou and L. Moussiades, "An overview of chatbot technology," in *Artificial Intelligence Applications and Innovations*, IFIP Advances in Information and Communication Technology, vol. 584, pp. 373–383, Springer, 2020. doi: 10.1007/978-3-030-49161-1_31.
- [2] B. A. Shawar and E. Atwell, "Chatbots: Are they really useful?," *LDV Forum*, vol. 22, no. 1, pp. 29–49, 2007.

- [3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186. doi: 10.18653/v1/N19-1423.
- [4] Adobe Systems, "PDF reference, sixth edition," 2006. [Online]. Available: https://www.adobe.com/devnet/pdf/pdf_reference.html. [Accessed: Jul. 13, 2025].
- [5] J. Guo, Y. Fan, Q. Ai, and W. B. Croft, "A deep relevance matching model for ad-hoc retrieval," in *Proc. ACM Int. Conf. Information and Knowledge Management (CIKM)*, 2016, pp. 55–64. doi: 10.1145/2983323.2983769.
- [6] A. Vaswani *et al.*, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008.
- [7] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP*, 2019, pp. 3982–3992. doi: 10.18653/v1/D19-1410.
- [8] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, 2021. doi: 10.1109/TBDATA.2019.2921572.
- [9] H. W. Chung *et al.*, "Scaling instruction-finetuned language models," *arXiv preprint arXiv:2210.11416*, 2022.
- [10] J. Weizenbaum, "ELIZA — a computer program for the study of natural language communication between man and machine," *Commun. ACM*, vol. 9, no. 1, pp. 36–45, 1966. doi: 10.1145/365153.365168.
- [11] Y. Zhang *et al.*, "Dialogpt: Large-scale generative pre-training for conversational response generation," in *Proc. ACL System Demonstrations*, 2020, pp. 270–278.
- [12] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Found. Trends Inf. Retr.*, vol. 3, no. 4, pp. 333–389, 2009. doi: 10.1561/15000000019.
- [13] T. Neumann, A. Becker, and M. Hofmann, "Design and evaluation of an LLM-powered chatbot for database education," in *Proc. ACM Conf. Learning at Scale*, 2025.
- [14] K. Alam, M. M. Rahman, and T. Akter, "A PDF-driven AI chatbot for project document management," *Int. J. Adv. Comput. Sci. Appl.*, vol. 15, no. 1, pp. 1–10, 2025.
- [15] W. Wu and J. Luo, "Challenges of AI chatbot adoption for SMEs: A resource allocation perspective," *J. Small Bus. Manage.*, vol. 63, no. 2, pp. 234–249, 2025.
- [16] A. Alsharhan, O. Alfarraj, and S. Alzahrani, "A systematic review of chatbot adoption in higher education institutions," *Educ. Inf. Technol.*, 2024.
- [17] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [18] K. V. N. Reddy *et al.*, "Proceedings of the 2025 IEEE Int. Conf. Interdisciplinary Approaches in Technology and Management for Social Innovation (IATMSI)," 2025. doi: 10.1109/IATMSI64286.2025.10985223.
- [19] B. Grueshma, M. Sireesha, and S. N. T. Rao, "Lecture notes in networks and systems," in *Lecture Notes in Networks and Systems*, 2022. doi: 10.1007/978-981-16-7657-4_3.
- [20] A. Seva, S. N. T. Rao, and M. Sireesha, "Proc. 13th IEEE Int. Conf. Communication Systems and Network Technologies (CSNT)," 2024. doi: 10.1109/CSNT60213.2024.10546170.
- [21] S. N. T. Rao *et al.*, "Proceedings of the 2025 IEEE Int. Conf. Interdisciplinary Approaches in Technology and Management for Social Innovation (IATMSI)," 2025. doi: 10.1109/IATMSI64286.2025.10984969.

CAICCIT__0243_An_Interactive_PDF_Chatbot_Using_Sema...



My Files



My Files



NUSRL, Ranchi

Document Details

Submission ID

trn:old::3618:12777961

Submission Date

Feb 8, 2026, 6:55 PM GMT+5:30

Download Date

Feb 8, 2026, 6:57 PM GMT+5:30

File Name

CAICCIT__0243_An_Interactive_PDF_Chatbot_Using_Semantic_Embeddings_and_FAISS_Based_Vect....pdf

File Size

2.3 MB

6 Pages

3,557 Words

22,626 Characters

7% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text
- Cited Text

Match Groups

-  **21 Not Cited or Quoted 7%**
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 5%  Internet sources
- 6%  Publications
- 6%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 21 Not Cited or Quoted 7%**
Matches with neither in-text citation nor quotation marks
- 0 Missing Quotations 0%**
Matches that are still very similar to source material
- 0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 5% Internet sources
- 6% Publications
- 6% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Submitted works	Bowie State University on 2025-12-17	<1%
2	Publication	"Proceedings of International Conference on Computing Systems and Intelligent ...	<1%
3	Internet	acadpubl.eu	<1%
4	Submitted works	University of Oklahoma on 2025-03-16	<1%
5	Submitted works	University of Birmingham on 2025-04-17	<1%
6	Internet	www.cdkeys.com	<1%
7	Submitted works	University of Massachusetts, Lowell on 2015-07-21	<1%
8	Publication	"Communication and Intelligent Systems", Springer Science and Business Media ...	<1%
9	Publication	Dimitris Mourtzis, John Angelopoulos, Angelos Nektarios Arvanitis, Nikos Panopo...	<1%
10	Submitted works	Imperial College of Science, Technology and Medicine on 2025-06-13	<1%

11	Submitted works	Kennesaw State University on 2025-12-10	<1%
12	Internet	thesai.org	<1%
13	Publication	Prashant, Monika Poriye, Pradeep Mittal, Naveen Sharma. "Automating Universit...	<1%
14	Internet	arxiv.org	<1%
15	Internet	www.mdpi.com	<1%
16	Publication	Chunduri, Pramod. "Enabling Semantically Richer Queries Over Unstructured Dat...	<1%
17	Internet	www.frontiersin.org	<1%
18	Submitted works	University of Stirling on 2024-09-03	<1%
19	Submitted works	University of Ulster on 2025-12-25	<1%